Bouncer: Competence Auditing for Set-Local Learned Microarchitectural Controllers

Anonymous authors
Paper under double-blind review

Abstract

Learned controllers—reinforcement-learning prefetchers, imitation-trained cache-replacement policies, neural branch predictors—are entering the processor datapath because they beat hand-tuned heuristics *on average*. That average is fragile. A learned controller's advantage is workload-dependent and can silently invert on a new workload, and a co-located adversary that knows the design can *steer* a shared controller into its bad regime while the aggregate performance counters still look normal. Nothing checks, at runtime, whether the controller is still beating the heuristic it replaced.

We present **Bouncer**, a runtime monitor and trust gate that measures *realized competence* directly. It repurposes *set-dueling*—already in commercial caches—to secretly run the learned controller against a safe fallback on a hidden, per-epoch-reshuffled slice of sets, compare the reward each earns, and revert to the fallback whenever the learned controller stops winning. Because the winner is *measured*, we prove a bounded-regret safety floor: over the union of audited declared domains, and in the controller's own reward, Bouncer's *expected* regret against the fallback is at most $N_{ep} \cdot D \cdot r_{\max} + \phi_P T_{att} r_{\max} + \alpha T c_{sw}$—a per-episode detection-and-re-trust transient, an audit-exposure term that grows only with the dueling fraction of attack duration, and a false-alarm term. Because the set assignment is a hardware *secret*, an attacker cannot tell which accesses are scored, yielding a per-domain mimicry-resistance property that follows exactly from that secrecy. The auditor is a few cheap always-on tripwires gating an expensive confirmer, fits in ~ 336 bytes ($\sim 0.13\%$ of a 256 KB SRAM) with no belief-state machinery, and by an analytical argument (RTL timing is future work) adds no latency on the cache-access critical path.

Our organizing result is *where this works*. A secret per-set audit can attribute reward to the right policy only for a *set-local* controller—one whose decision and reward fall on the same set (canonically cache *replacement*; PC-indexed predictors are *expected* set-local but untested). *Prefetching is the cautionary boundary*, since a prefetch benefits a different set than it was decided on. On real hardware a second condition appears: the policy *contrast* must survive the reshuffle (be *reseed-identifiable*), because the same secret reshuffle that buys mimicry resistance erases the policy-specific state a path-dependent reward accumulates. A (near-)*stateless* reward suffices but is not necessary—we exhibit a stateful reward whose contrast is state-invariant and stays auditable. **Set-locality is therefore necessary but not sufficient: a secret, reseeded audit also needs a reseed-identifiable contrast**; the two conditions govern which controllers Bouncer can certify (we argue each is necessary and demonstrate the boundary; a full sufficiency theorem is future work).

We validate the guarantees in a fully-released *synthetic competence harness* whose i.i.d. per-decision reward meets both conditions. The estimator tracks ground truth (R^2=0.997), the steady-state floor stays within 0.64% of the fallback under attack, detection costs one audit window, and the clean-workload tax is 0.35%. The headline (over 50 seeds): under an adaptive mimicry adversary *within a declared, secret-intact domain*, an input-distribution monitor detects *none* of the attacks (0/50, Wilson 95% upper bound 0.07) while Bouncer detects *all* (50/50, lower bound 0.93), because it scores realized competence where an input monitor is blind to it by construction. A separate leak ablation isolates what the sampler's

secrecy adds: against a leak-aware regional adversary, detection holds up to a moderate leak and then collapses sharply (in the 0.6–0.8 leaked-fraction band, well before the assignment is fully known)—while a sub-resolution *slow-bleed* adversary evades the audit's resolution altogether (a named, open vulnerability). We then probe the boundary on real ChampSim deployments on SPEC CPU2017. As an L1D prefetcher module Bouncer caps a corrupted prefetcher at the prefetch-off floor but over-gates a genuinely-helpful one, and swapping to the controller's own reward does *not* fix it, because a prefetcher's reward cannot be localized to a set. As an LLC *replacement* module it *is* set-local, yet exposes the second condition: a cache's reward is path-dependent, so its policy contrast is not *reseed-identifiable* and the secret reseed that buys mimicry resistance *confounds* the per-window estimate (fixed leaders resolve it; the prior "$\hat{\Delta} \approx 0$" was a missing-callback software artifact, now fixed). The synthetic results are the quantitative claims; the ChampSim study is a real-simulator *boundary* study, not a clean-case validation or a full sweep.

1 Introduction

Modern processors increasingly hand small *learned* models control of decisions that used to be made by fixed, hand-tuned rules. Consider the *cache*—a small, fast memory that holds recently or soon-to-be-used data so the processor rarely waits on slow main memory. Two decisions dominate a cache's performance: which data to *prefetch* (fetch early, before it is asked for) and which block to *evict* when the cache is full (*replacement*). Learned versions of both now beat the classic heuristics—a reinforcement-learning prefetcher Bera et al. (2021), an imitation-learned replacement policy Jain & Lin (2016); Shi et al. (2019)—and similar learned controllers are arriving for memory scheduling and branch prediction Ipek et al. (2008); Jiménez & Lin (2001). The tempting move is to ship them and trust the average: they were validated to win, so let them run.

"Wins on average," though, is a property of the workloads a controller was validated on, and it does not travel. On a new program or a new execution phase a learned controller can quietly become *worse* than the simple heuristic it displaced—a silent regression that aggregate performance counters never flag. Worse still, a shared on-chip controller is exercised by *every* program on the core, so a hostile co-running program can *steer* it: craft memory accesses that push the controller into its bad regime while keeping the aggregate counters looking normal Guo et al. (2022); Evtyushkin et al. (2018). Nothing in a deployed system checks, at runtime, whether the learned controller is still earning its keep. The obvious check—watch the controller's *inputs* for distribution shift—is both insufficient and unsafe: an attacker can hold the input statistics in-distribution while wrecking the controller's behavior (a *mimicry* attack), and an input monitor is blind to that by construction.

We instead check the thing that actually matters: *realized competence*, whether the learned controller is beating the fallback *right now*—and we show this can be done cheaply, provably, and without trusting the inputs. The mechanism (Figure 1) builds on the fact that a cache slices its storage into many *sets* (address buckets that each memory address maps into). It repurposes *set-dueling*—running two candidate policies on different sets and comparing them—a technique already in commercial caches: we secretly dedicate a randomly chosen handful of sets to run the learned controller and another handful to run the safe fallback, compare the reward they actually earn, and let the rest of the cache follow whichever is winning. If the learned controller stops winning, the gate reverts to the fallback. Two properties turn this from a heuristic into a guarantee. Because the winner is *measured*, not predicted, we prove a floor: the system's *expected* regret against the fallback, over any run, is at most a small, quantified amount. And because the set assignment is a hardware *secret*, reshuffled each epoch, an attacker cannot tell which accesses are being scored—so it cannot look good on the hidden test while degrading the victim.

The central finding is *where this works and where it does not.* Secret set-dueling can attribute reward to the right policy only if that reward is *set-local*: the outcome of a decision made on a set is realized on that same set. Cache *replacement* is set-local—an eviction on a set is scored by the hits and misses on that set—and is the clean case. *Prefetching is the instructive boundary*: a prefetch triggered on one set benefits a *different* set, so no per-set reward captures it, and we confirm on a real simulator that not even the controller's

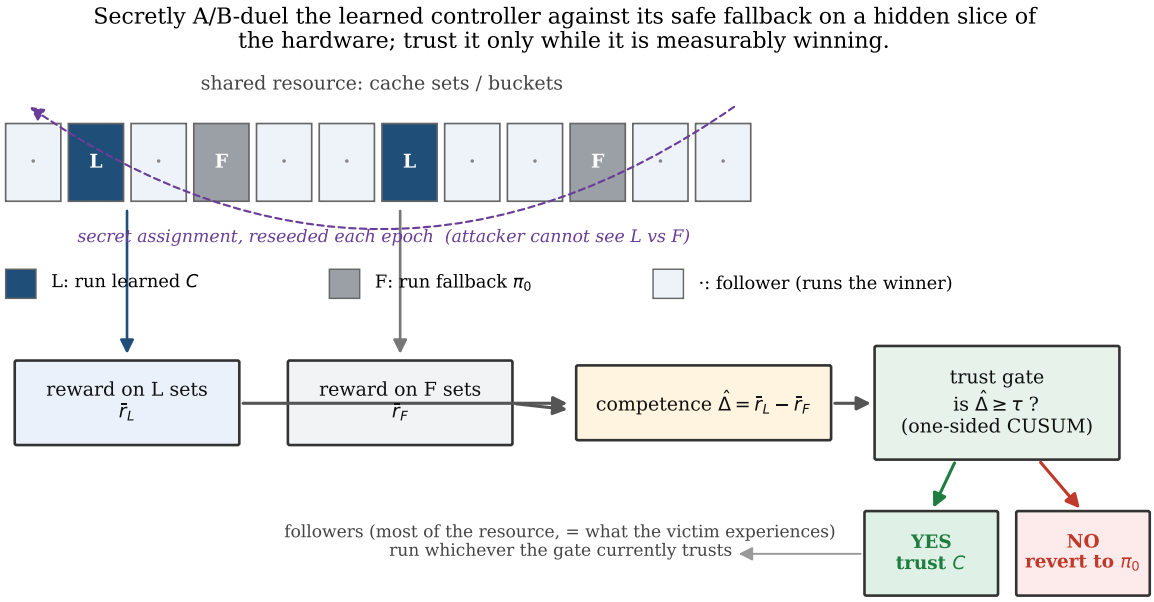


Figure 1: **Secretly A/B-duel the learned controller against its safe fallback on a hidden slice of the hardware; trust it only while it is measurably winning.** A few randomly chosen *sets* (address buckets in the cache) run the learned controller, a few run the fallback, and the assignment is a hardware secret reshuffled each epoch (a fixed interval). Because the winner is *measured*, the system's expected regret against the fallback is provably bounded; because the assignment is *secret*, an attacker cannot look good on the hidden test while degrading the victim.

own reward rescues the audit (Section 11.13). A second condition surfaces on real hardware, and it is *not* statelessness. The secret reshuffle erases any long-run, policy-specific state a reward accumulates, so what the audit needs is that the policy *contrast*—the C-minus-fallback advantage—survive that erasure: the reward must be *reseed-identifiable*. A (near-)stateless reward is the clean way to get this, but not the only way: we show an additive-warmup reward that is *stateful* yet stays auditable because its contrast is state-invariant, and a multiplicative-warmup reward (the cache case) that fails because its contrast scales with the state the reshuffle destroys. Together, set-locality and reseed-identifiability are the two conditions we identify as *governing* which controllers this kind of audit can certify: we argue each is necessary and demonstrate the boundary on real and synthetic controllers, leaving a complete sufficiency theorem to future work—the paper's organizing result, with prefetching and one named security limitation as its honest boundaries.

Contributions.

1. **Two conditions for faithful auditing** (Section 2, Definition 3, Remark 1): set-locality and reseed-identifiability of the policy contrast are two conditions we identify as governing when secret set-dueling can faithfully audit a learned controller—we argue each is necessary and demonstrate the boundary, leaving a full sufficiency theorem to future work (statelessness is one sufficient special case of the second). On a real simulator we probe the boundary one trace at a time: replacement is set-local (and we expose where its stateful reward is not reseed-identifiable), while prefetching is not set-local.

2. **The mechanism** (Section 5): a counterfactual-free competence estimator that repurposes the set-dueling substrate Qureshi et al. (2007); Jaleel et al. (2010) from *choosing* a policy to *trusting* one, gated by a change detector and a four-state trust machine, all off the memory-access critical path (out of the latency-critical access loop).

3. **Two guarantees** (Section 7): an *expected*-regret safety floor (Lemma 1)—bounding the mean regret against the fallback—tied to the run-length theory of the detector, and a mimicry-resistance result (Proposition 1) that follows exactly from the secrecy of the sampler; together with an honestly named limitation, a slow-bleed attacker who stays below the audit's resolution and evades it.

4. **A cheap, released design** (Sections 11 and 12): a two-tier microarchitecture whose auditor fits in a few hundred bytes and sits off the cache's critical path, evaluated by a synthetic harness that validates the guarantees and a study on a real cycle-accurate CPU simulator (ChampSim) that both demonstrates the downside cap and exposes the reward-localizability boundary.

We are explicit throughout (Sections 10 and 14) about what is *proven*, what is *measured* in a controlled synthetic harness—whose load-bearing assumptions (bounded reward, a partitionable resource, and a secret uniform sampler, A1–A6; plus, for faithful auditing, set-locality and reseed-identifiability) are exactly those the guarantees rest on—and what is *demonstrated* on a real simulator.

2 Formal setting

A learned controller C is invoked at decision points $t = 1, 2, \ldots$. At each it observes microarchitectural state (the hardware's internal counters and flags) $x_t \in \mathbb{R}^d$, emits action $a_t \in \mathcal{A}$, and after a short horizon the environment yields a *bounded* reward $r_t \in [0, r_{\max}]$ (prefetch hit×timeliness, cache hit, −latency, branch correct). C was trained/validated on a distribution D_{val} of (x, a, r). A fixed safe fallback π_0 has a known, attack- resistant competence floor.

Definition 1 (Competence advantage)**.** *Over a window W, $\Delta_W = \frac{1}{|W|} \sum_{t \in W} \left(r_t^C - r_t^{\pi_0} \right)$.*

$r_t^{\pi_0}$ is counterfactual—one cannot run both policies on the same decision—which is the central estimation challenge Section 5 solves.

Definition 2 (Off-policy condition)**.** *The system is* off-policy *at W iff $\Delta_W < \tau$ for a trust threshold $\tau \geq 0$. τ=0 means "learned no longer beats the floor"; τ>0 gives margin and hysteresis.*

This one scalar unifies the two threats: an adversarial push toward C's decision boundary and a benign distribution shift both manifest as Δ_W falling. The auditor detects the condition; a lightweight triage (Section 8) guesses the cause to pick the response.

Definition 3 (Set-locality)**.** *Let the resource be partitioned into dueling sets $\{s\}$. A controller is* set-local *if the reward r_t for a decision made on set s is realized on set s (so it can be attributed to s's pool). Then the per-pool means $\bar{r}_L, \bar{r}_F$ are unbiased estimates of each policy's competence on the sampled sets, and $\hat{\Delta} = \bar{r}_L - \bar{r}_F$ is a faithful competence estimate.*

Set-locality is the enabling condition for the entire mechanism: it is precisely what makes $\hat{\Delta}$ (Section 5) a clean signal rather than a mis-attributed one. Cache *replacement* (deciding which cached block to evict) satisfies the spatial-attribution requirement exactly (the eviction on s is rewarded by hits/misses on s); PC-indexed predictors satisfy it too—though, as Remark 1 and Section 11.13 show, replacement's reward is *stateful*, so a secret reseeded audit does not recover a faithful $\hat{\Delta}$ there at per-window grain; *prefetch* violates it (a prefetch on s benefits a different set s'), and we show in Section 11.13 that no per-set reward—not even the controller's own— recovers a faithful $\hat{\Delta}$ for it. We therefore present Bouncer as a mechanism for the *set-local* class, with replacement as the canonical instance and prefetching as the demonstrated boundary.

Remark 1 (Set-locality is necessary, not sufficient—a reseed-identifiability condition)**.** *The unbiasedness in Definition 3 can still fail when the reward is* stateful*—e.g. a cache hit on s depends on which blocks prior evictions on s retained. Reseeding the leader assignment each epoch (the source of mimicry resistance, Proposition 1) keeps every sampled set in its short-run, cold state, so the estimator sees the contrast $r_C - r_F$ under the reseed-induced state distribution $\mathcal{D}_{rs}$ rather than at steady state. The audit is faithful when the* signed *reseed contrast $\hat{g} := \mathbb{E}_{s \sim \mathcal{D}_{rs}}[r_C(s) - r_F(s)]$ lands on the same side of the trust threshold as the true competence gap Δ—formally, when $\operatorname{sign}(\hat{g} - \tau) = \operatorname{sign}(\Delta - \tau)$ with a margin $|\hat{g} - \tau| \geq \gamma/2$ large enough to clear the CUSUM reference K, so the gate's τ-decision is preserved. Call this* reseed-identifiability*. The*

signed *form matters: a state-dependent contrast that reseeding drives to the wrong side of* τ *(e.g. a true gap* $+0.3$ *read as* -0.3*) would reverse the gate, which is exactly the failure this rules out. It is* not *statelessness. A (near-)stateless reward satisfies it trivially, but so does any reward whose* contrast *is state-invariant: an additive warmup* $r_C(k) = \mu_C - be^{-ck}$, $r_F(k) = \mu_F - be^{-ck}$ *has contrast* $\mu_C - \mu_F$ *at every state* k *and stays auditable despite being stateful, whereas a* multiplicative *warmup* $r_C(k) = \mu_C(1 - e^{-ck})$ *has contrast* $(\mu_C - \mu_F)(1 - e^{-ck})$ *that the reseed attenuates to noise (Section 11.7). Our synthetic harness's i.i.d. per-decision reward (A1) is stateless and so trivially reseed-identifiable; real cache replacement is set-local but its reward behaves like the multiplicative case, so the reseeded audit collapses while fixed leaders recover the gap (Section 11.13). A secret, reseeded competence audit thus requires set-locality* and *a reseed-identifiable contrast; a (near-)stateless reward is the clean sufficient case.*

3 Threat model

In scope. A co-located tenant/process sharing C that issues arbitrary memory accesses and knows the Bouncer design (Kerckhoffs); its goal is to DoS the victim or steer C into a covert/side channel. Online-learning poisoning is an extension—the GATED state freezes learning. **Out of scope.** Physical/fault attacks and direct weight tampering (the base model assumes weights fixed at deploy). **Online-learning poisoning is explicitly out of scope for the base model:** an attacker who corrupts C's *weights* during TRUSTED operation (the phase where the learned controller is in charge) so that C looks competent on the secret leaders but harms the victim is not addressed here—freezing learning at GATED stops further poisoning but does not undo it, and PROBING re-trust would restore a poisoned policy. Defending learned *state* (vs. the steered-*input* threat we do treat) is future work. **The secret.** A per-epoch RNG seed assigning the dueling sets (which cache sets are secretly audited), never exposed through any architectural channel.

The adaptive adversary (headline). The headline attack is *mimicry*: the adversary keeps the Tier-A statistics (the input-monitoring signals) in-distribution while degrading C. Two distinct defenses apply, and we are careful not to conflate them. Because Bouncer scores *realized competence* rather than inputs, this attack is caught regardless of secrecy—an input monitor's failure against it is *definitional* (Section 11.5), which is what the 50/50-vs-0/50 gap measures. *Secrecy* defends the next move: an attacker who could *identify* the dueling sets would spare them and pass the audit, so the randomized, reseeded sampler is what keeps that move out of reach (Proposition 1); the leak ablation quantifies exactly how much secrecy buys as the assignment leaks.

Realizability—what we do and do not demonstrate. Bouncer's guarantees bound the cost of *any* in-domain competence degradation *however it arises*: benign distribution shift or adversarial steering are the same scalar event ($\Delta_W < \tau$) to the auditor. We realize the mimicry/steering adversary in the synthetic competence harness, where the latent stress u is the controllable knob; the real-ChampSim study (Section 11.13) drives the auditor with an *injected, controlled, labelled* competence degradation—a deliberately-bad eviction policy active on a known phase—*not* a demonstrated end-to-end real-silicon steering attack. Whether a co-located tenant can realizably steer a *specific deployed* controller to sub-resolution harm on real silicon, rather than in our harness, is *not established here and is future work*; the defense's guarantee is degradation-source-agnostic, so the harness's injected degradation is a faithful stand-in regardless of how a real degradation would be induced.

4 Architecture

Bouncer is two-tier because cheap signals are spoofable and the unspoofable signal is expensive (Figure 2). Figure 2 shows the layout: cheap always-on tripwires (Tier-A) feed a read-only tap and gate an expensive competence confirmer (Tier-B), with nothing on the datapath critical path (the timing-limiting logic that sets clock speed). The cheap tripwires run always; they gate the expensive confirmer.

Gate state machine. The gate is a four-state machine that decides where the learned controller C runs. *TRUSTED*: C active everywhere, Tier-A on, Tier-B on a slow background duty cycle (fraction of time it runs). Any Tier-A CUSUM (a cumulative-sum change detector) firing $\rightarrow$ *SUSPECT*. *SUSPECT*: C still active, Tier-B duty cycle raised (more dueling sets, shorter window); the Tier-B $\hat{\Delta}$ CUSUM firing $\rightarrow$ *GATED*;

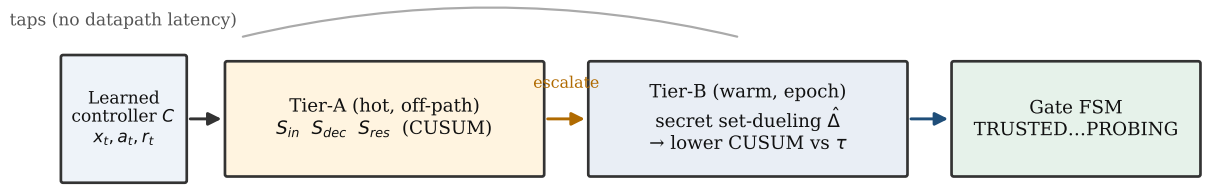


Figure 2: Two-tier auditor. Cheap always-on Tier-A tripwires gate an expensive Tier-B competence confirmer; taps are read-only and nothing sits on the datapath critical path.

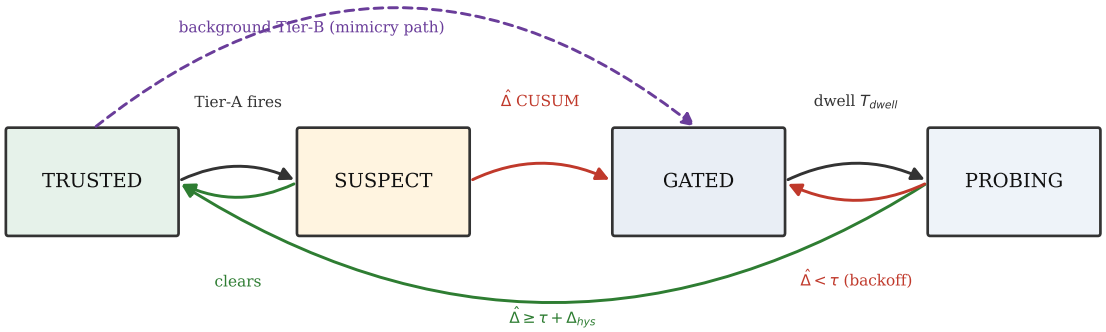


Figure 3: The trust-gate FSM. Hysteresis and a minimum dwell prevent thrash; PROBING makes re-trust measured, not timed. The dashed path lets background Tier-B gate a pure-mimicry attack that keeps Tier-A silent.

Tier-A clearing for T_{clear} → TRUSTED. *GATED*: π_0 active everywhere *except the* n_L *Leader-L sets, which keep running* C *so* $\hat{\Delta}$ *stays measurable*, online learning frozen, optional attribute/throttle; after a dwell T_{dwell} → *PROBING*. *PROBING*: C re-enabled on a small audited region only, Tier-B measures $\hat{\Delta}$ there; $\hat{\Delta} \geq \tau + \Delta_{hys}$ for $T_{reprobe}$ → TRUSTED, else → GATED with exponential backoff on T_{dwell}. Hysteresis (τ to enter, $\tau + \Delta_{hys}$ to leave) plus a minimum dwell prevent thrash; PROBING makes re-trust *measured*, not timed, defeating an attacker who waits out a fixed cooldown.

A design subtlety the build surfaced (Figure 3): a *pure* mimicry attack keeps Tier-A silent, so if SUSPECT were the only route to GATED the attack would evade the gate. We therefore allow the background Tier-B CUSUM to drive TRUSTED → GATED directly; Tier-A escalation only *accelerates* detection by raising Tier-B's duty cycle. The two tiers trade latency/energy, not correctness.

5 Tier-B: competence via randomized-secret set-dueling

Mechanism. Partition the controller's resource (cache sets, memory banks, PC-hash buckets) into three pools, reseeded each epoch (a fixed-length run of decisions) from a hardware RNG (random-number generator) whose assignment is unobservable to software: *Leader-L* (n_L sets, always run by C), *Leader-F* (n_F

sets, always run by π_0), and the *Follower* rest (run whichever leader currently wins—what the victim mostly experiences). Set-dueling—racing two policies on small dedicated sample pools—is already in silicon Qureshi et al. (2007); Jaleel et al. (2010); Bouncer repurposes it from policy selection to policy trust. The estimator is

$$\hat{\Delta}_W = \bar{r}_L - \bar{r}_F, \tag{1}$$

the mean reward rate on the sampled L vs F sets over window W. Because Leader-L sets *ran* C and Leader-F sets *ran* π_0, $\hat{\Delta}$ is a *measured*, non-counterfactual quantity.

Concentration → detection latency (the key tradeoff). With m decisions per set per window and bounded rewards, $\mathrm{Var}(\bar{r}_{\mathrm{pool}}) \le r_{\max}^2/(4mn)$, so

$$\mathrm{Var}(\hat{\Delta}_W) \;\lesssim\; \frac{r_{\max}^2}{4m}\Big(\tfrac{1}{n_L} + \tfrac{1}{n_F}\Big) \;=:\; \sigma_\Delta^2. \tag{2}$$

To resolve a competence gap of size τ one needs $\sigma_\Delta \ll \tau$. Smaller pools or shorter windows give a noisier $\hat{\Delta}$, a higher CUSUM threshold H to hold the false-alarm rate, a longer detection delay D, and a larger safety-floor slack (Lemma 1). This chain—*pool size → estimator variance → CUSUM threshold → detection delay → regret floor*—is the central design dependency; every hyperparameter lives on it.

Detector. A one-sided lower CUSUM (cumulative-sum change detector) on $\hat{\Delta}_W$ catches a downward shift below τ:

$$C_t^- = \max\big(0,\, C_{t-1}^- + (K - \hat{\Delta}_W)\big), \quad K = \tau + \tfrac{\gamma}{2}, \tag{3}$$

alarming when $C_t^- > H$, and resetting on alarm (repeated-CUSUM, so it re-arms for the next change rather than latching after a long excursion). Standard quickest-detection theory Lorden (1971); Moustakides (1986) motivates the CUSUM as the near-optimal detector for a downward shift under a false-alarm constraint (optimality is exact only for a known post-change law; here $\hat{\Delta}_W$ is an estimated, variance-inflated statistic, so we treat it as a well-motivated detector); we work the ARL (average run length between false alarms)/delay constants in Section 7.

6 Tier-A: cheap tripwires

All Tier-A signals update on a sampled $1/k$ of decisions, a few adds/compares each, in $\le$ low-KB SRAM (a few kilobytes of on-chip memory), and *never gate on their own*—they only spend Tier-B budget, so their false positives cost a little energy, never correctness.

S_{in} **(input covariate shift).** A frozen reference sketch of D_{val} features: a random projection $P \in \mathbb{R}^{d\times p}$ ($p{\approx}8$) feeding per-component quantile sketches; online, a windowed energy distance vs the reference. Cheap, high-sensitivity, and *spoofable*—the adversary can hold marginals fixed—which is exactly why it cannot be the trust signal.

S_{dec} **(decision-confidence collapse).** If C exposes value/Q/logits, monitor action-entropy collapse, max-Q margin, and (for online learners) temporal-difference-error magnitude (TD-error, the reinforcement-learning surprise signal). Pythia's reward and the confidence signals of SHiP and perceptron (two learned cache/branch controllers) are already computed in silicon Wu et al. (2011); Jiménez & Lin (2001), so S_{dec} is nearly free there.

S_{res} **(innovation residual).** A 16-weight linear forward model $g : (x_t, a_t) \to \hat{r}_t$ fit on D_{val}; online residual $e_t = r_t - \hat{r}_t$; two-sided CUSUM (a cumulative-sum drift detector) on e_t. This catches "the world responds differently than at validation" even when input marginals look clean—the architectural analogue of a Kalman innovation (the gap between a predicted and an observed signal). It is the signal that separates a competence-aware monitor from an input-only one.

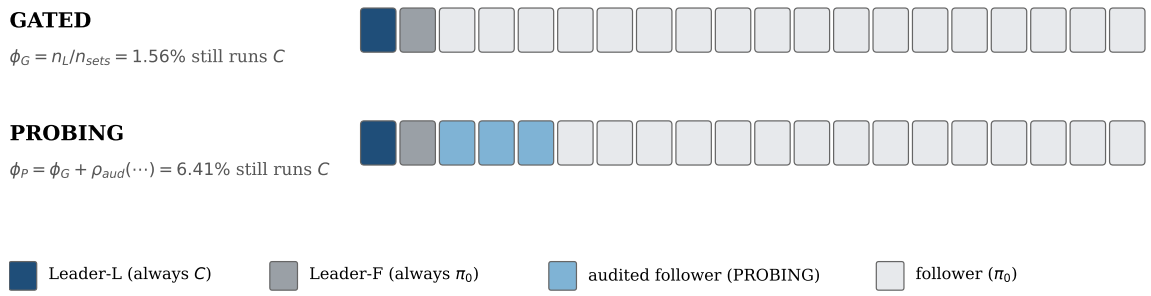


Figure 4: Which sets keep running C under attack (schematic, not to scale). Leader-L and Leader-F never change; PROBING additionally re-enables C on a ρ_{aud} audited follower fraction. This is exactly ϕ_G and ϕ_P of Lemma 1's audit-exposure term.

7 Guarantees

7.1 Safety floor / bounded regret

Assumption 1. (A1) *per-decision reward* $\in [0, r_{\max}]$*;* (A2) *a* drop episode *is a* maximal run *of consecutive windows with true competence* $\Delta_t < \tau$ *(*$K = \tau + \gamma/2$*), and over such an episode the* expected number of fully-open windows*—*C *running everywhere (TRUSTED/SUSPECT), the initial detection delay* plus any false re-trust excursions*—is* $\leq D$*. We treat this as an* assumption on the FSM that we characterize empirically, not a closed-form bound*: Section 11.12 measures* D through the real controller *and finds* $D \approx$ *the initial delay (*$\approx H/(K - \Delta_t)$*, Lorden/Siegmund) with* ≈ 0 *false re-trusts for the reseeded, near-i.i.d. audit (fully-open fraction* ≤ 0.07*). It is* not *horizon-independent in general: a temporally-*correlated *(stateful) audit strings together the* $T_{reprobe}$ *crossings a false re-trust needs, inflating* D*—the same reseed-identifiability boundary—which we name as a limitation rather than bound;* (A3) *the per-window false-alarm* probability *is* $\leq \alpha = 1/\mathrm{ARL}_0$ *(a marginal bound;* no *independence across windows is assumed);* (A4) *a gate switch transient costs* $\leq c_{sw}$*;* (A5) *there are* N_{ep} *drop episodes spanning* $T_{att} \leq T$ *windows over a horizon of* T *windows;* (A6) *the sets that keep running* C *during a drop are chosen by the* secret *sampler—the* n_L *Leader-L sets always, plus in PROBING a* uniform *secret subset (fraction* ρ_{aud}*) of the followers, redrawn each epoch—so every set carries* C *with probability* $\leq \phi_P$ independent of its traffic *(*ϕ_G *while GATED,* ϕ_P *while PROBING).*

Audit exposure (from the construction). A fixed slice of the resource keeps running the controller C even during a drop, and this slice sets the audit-exposure cost. The n_L Leader-L sets run C in *every* gate state (they must, so $\hat{\Delta}$ stays measurable and PROBING re-trust is *measured* rather than timed; Section 5), and PROBING additionally re-enables C on a *uniform secret* fraction ρ_{aud} of the followers—drawn by the same secret sampler and redrawn each epoch, so a set's exposure does not depend on its index or traffic (a fixed, e.g. sorted, audit subset would violate this; Section 11.11). So the share of the resource still running C during a drop is $\phi_G = n_L/n_{sets}$ while GATED and $\phi_P = \phi_G + \rho_{aud}\left(1 - \frac{n_L + n_F}{n_{sets}}\right)$ while PROBING; at the pinned operating point these shares are small (ϕ_G=1.56%, ϕ_P=6.4%). Let T_{att} be the number of windows inside drop episodes.

Lemma 1 (Safety floor, in expectation)**.** *Under Assumption 1, for an* arbitrary *per-set traffic distribution, the expected regret against the fallback—over the secret sampler, the detection delay, and the false-alarm events—obeys*

$$\mathbb{E}\Big[\sum_t \big(r_t^{\pi_0} - r_t^{\text{Bouncer}}\big)\Big] \;\leq\; \underbrace{N_{ep}\, D\, r_{\max}}_{\text{detection + re-trust}} + \underbrace{\phi_P\, T_{att}\, r_{\max}}_{\text{audit exposure}} + \underbrace{\alpha\, T\, c_{sw}}_{\text{false alarm}}. \tag{4}$$

Every term is an expected value, and the bound uses only the marginal *assumptions A2, A3, A6 and linearity of expectation—*no *independence across windows. The middle term is the price of continuous,* measured *auditability; it grows only at the dueling fraction ϕ_P of the attack duration, not at full resource.*

Proof sketch. Partition windows and take expectations termwise. *(i) Audit exposure.* During a drop the sets still running C are the Leader-L sets (GATED) plus, in PROBING, the uniform secret audit subset. Their regret is the *traffic* on those sets, *not* their count; A1–A5 alone do not bound per-set traffic, so on any single window a hot set tagged for C can cost a full $r_{\max}$ (Section 11.11). A6 is what controls it: each set carries C with probability $\leq \phi_P$ *independent of its traffic*, so for any fixed per-window traffic vector w ($\sum_s w_s = 1$), $\mathbb{E}[\text{exposure}] = \sum_s w_s \Pr[s \text{ runs } C]\, r_{\max} \leq \phi_P r_{\max}$; summing over the T_{att} drop windows gives $\phi_P T_{att} r_{\max}$. *(ii) Detection and re-trust.* Within an episode C runs everywhere on its *fully-open* windows—the initial detection delay *plus* any false re-trust excursions; by A2 their expected count is $\leq D$, so the expected fully-open regret is $\leq N_{ep} D r_{\max}$ (linearity over episodes). This is where a false re-trust is charged: A2's D counts it, and Section 11.12 measures it (small in the reseeded regime; inflated by a correlated audit). *(iii) False alarms.* By A3 the expected number of false alarms is $\sum_{t=1}^{T} \Pr[\text{alarm}_t] \leq \alpha T$ (linearity; marginals suffice, no independence), each costing $\leq c_{sw}$. *(iv)* Trusted windows with $C \geq \pi_0$ contribute non-positive regret. Summing (i)–(iii) yields eq. (4). □

A high-probability refinement, for the exposure term only. Because the assignment is redrawn *independently* each epoch, the per-window exposures are independent $[0, r_{\max}]$ variables, so Hoeffding tightens the middle term to a high-probability envelope: with probability $\geq 1 - \delta_s$,

$$\textstyle\sum_t (\text{exposure regret})_t \;\leq\; \phi_P T_{att} r_{\max} \;+\; r_{\max}\sqrt{\frac{T_{att}}{2}\ln\frac{1}{\delta_s}}, \tag{5}$$

an $o(T_{att})$ slack that vanishes per-window over a sustained attack (empirically the envelope covers the worst-case concentrated-traffic total with coverage 1.0; Section 11.11). We do *not* claim analogous high-probability forms for the detection and false-alarm terms: the former would need a per-episode delay-*tail* bound (A2 supplies only the mean) and the latter a martingale/renewal concentration for the *stateful* CUSUM (A3 supplies only the marginal), neither of which we assume. The guarantee we assert is the expected-regret bound eq. (4).

Two of the three regret terms are tunable, and one is fixed by design. The detection and false-alarm slack are the Section 5 knobs, while the audit-exposure term is instead fixed by the design constant ϕ_P. In the reseeded regime where false re-trust ≈ 0 (Section 11.12), A2's D reduces to the initial delay $D \approx H/(K - \Delta_{true})$, and the false-alarm rate is $\alpha = 1/\text{ARL}_0$, where ARL_0 (average run length to a false alarm) comes from Siegmund's corrected- boundary approximation Siegmund (1985), a standard sequential-analysis result. For a one-sided CUSUM (a running change detector) accumulating $(K - \hat{\Delta})$ with standardized drift $\delta = (K - \mu)/\sigma_\Delta$ and boundary $b = H/\sigma_\Delta + 1.166$,

$$\text{ARL}(\delta) \;\approx\; \frac{e^{-2\delta b} + 2\delta b - 1}{2\delta^2}. \tag{6}$$

In-control ($\mu \gg K$, $\delta < 0$), ARL_0 grows *exponentially* in H/σ_Δ, so false alarms are rare; out-of-control ($\delta > 0$), the delay is short. The guarantee thus degrades gracefully and *quantifiably* with detection delay and false-alarm rate.

7.2 Mimicry resistance

We state the property *conditionally*—it is exactly as strong as the secrecy and the within-domain exchangeability it assumes, and we are explicit that a *global* $\hat{\Delta}$ carries *no* guarantee against a redistributive attack.

Proposition 1 (Mimicry resistance, per-domain)**.** *Fix a contested domain R (e.g. a victim co-tenant's cache footprint) and run the dueling pools* within *R each epoch (a fixed audit window): draw Leader-L and the follower sets of R uniformly at random from R, unobservable to the adversary. Assume the within-R set outcomes are exchangeable up to a heterogeneity bias $|b_R| \leq \beta$ (an explicit assumption; the misspecification sweep of Section 11 is its empirical proxy). If the adversary holds all aggregate Tier-A*

statistics at their validation values, then any input-only strategy that keeps $\hat{\Delta}_R = \bar{r}_L - \bar{r}_F \geq \tau$ *forces* $\mathbb{E}[$*reward on the R followers*$] \geq r^{\pi_0} + \tau - \beta$*:* the victim cannot be degraded below the floor while evading the gate. *Under a leaked fraction* f *of the within-R assignment, the adversary may spare* $f\,n_L$ *Leader-L sets, biasing* $\hat{\Delta}_R$ *upward by* $\leq f\,(\bar{r}^{clean} - \bar{r}^{att})$*; the* unprotected $(1-f)$ *of the degradation still enters* $\hat{\Delta}_R$*, so the gate fires once the unprotected degradation pushes* $\hat{\Delta}_R$ *below the CUSUM reference* $K = \tau + \gamma/2$*—i.e.* $(1-f)(\bar{r}^{clean} - \bar{r}^{att}) \gtrsim \Delta_R^{clean} - K$*, where* Δ_R^{clean} *is the* un-attacked *competence advantage on R (up to* $O(\sigma_{\Delta,R})$ *estimator noise, a realized per-window event; empirically the crossing falls between* f=0.6*, where* $\hat{\Delta}_R \approx 0.084 < K$ *and detection holds, and* f=0.8*, where* $\hat{\Delta}_R \approx 0.227 > K$ *and it evades), i.e. detection is intact while the leaked fraction stays modest (empirically up to* $f \approx 0.6$*, Section 11.5) and collapses sharply in the* 0.6–0.8 *band, well before* f=1 *(not a gradual roll-off toward* 1*).*

Proof sketch. Within R the Leader-L and follower sets are an exchangeable partition under the secret uniform draw, so their reward means satisfy $\mathbb{E}[\bar{r}_L] = \mathbb{E}[\bar{r}_{\text{foll}}] + b_R$ with $|b_R| \leq \beta$. An adversary blind to the partition cannot apply a set-dependent perturbation distinguishing them, so any input-only strategy raising $\bar{r}_L$ raises the follower mean to within β. Evasion ($\hat{\Delta}_R \geq \tau$) thus forces follower reward $\geq r^{\pi_0} + \tau - \beta$. Leaked knowledge of fraction f lets the adversary exempt $f\,n_L$ Leader-L sets; the remaining $(1-f)$ fraction carries the degradation into $\hat{\Delta}_R$, detectable once the unprotected degradation exceeds the clean margin, $(1-f)(\bar{r}^{\text{clean}} - \bar{r}^{\text{att}}) \gtrsim \Delta_R^{\text{clean}} - K$ (up to $O(\sigma_{\Delta,R})$ noise). □

What this does and does not cover. (i) *Global* $\hat{\Delta}$ *is not enough.* A purely redistributive adversary that degrades the victim region while holding the *global* mean fixed evades a global $\hat{\Delta}$ outright; only per-domain dueling (the construction above, used in Section 11.5) detects it. (ii) *Heterogeneity bias.* Real cache sets / PC-buckets are not i.i.d. (hot/cold sets, conflict skew), so $\beta > 0$; the guarantee degrades by β. Secret *reseeding* each epoch shrinks this bias toward the i.i.d. value by re-randomizing which physical cache sets act as leaders. (iii) *Secrecy is load-bearing.* The whole argument rests on the adversary being blind to the within-R assignment—which is why the secrecy ablation (Section 11.5) is the headline experiment.

The coverage hole (an explicit slow-bleed attack on the budget). A budgeted auditor has a finest resolvable competence drop, and an adversary can hide harm below it forever. The proposition is genuinely scoped, not merely hedged: it covers a *declared* domain audited at finite resolution. From the concentration bound, the within-R gap resolvable with leader density ρ is $\delta_R = k\,\sigma_{\Delta,R} = k\,r_{\max}/\sqrt{2m\rho|R|}$, and—crucially—the leader count to audit at resolution δ_R is $n_{L,R} \geq k^2 r_{\max}^2/(2m\,\delta_R^2)$, *independent of* $|R|$. So under a fixed budget B there is a finest resolution $\delta_R^{\min}(B)$, and a redistributive adversary can pick a region small enough (or trickle harm slowly enough) that its per-window competence drop stays *below* $\delta_R^{\min}(B)$ forever—an undeclared, sub-resolution "coverage hole" that the safety-floor lemma, proven only over the *union of audited domains*, does not bound. This is the same hole the secrecy ablation walks on its f axis (TPR $1.0 \to 0$ as $f \to 1$). *We therefore claim mimicry resistance only for declared domains audited at resolution* $\delta_R^{\min}(B)$ *with an intact secret—not general, any-region security*; the slow-bleed attack is a real, un-closed vulnerability that bounds the contribution honestly. Figure 5 demonstrates it against the deployed auditor: a constant per-window harm kept below $\delta_R^{\min}$ (h=0.06) bleeds a victim region for the entire horizon—cumulative harm grows monotonically and unboundedly in T— while the measured gap $\hat{\Delta}_R$ holds above the trust threshold ($0.205 \geq \tau$) and the CUSUM (the cumulative-sum drift detector) never fires.

8 Attack-vs-drift triage

Attack and drift both trip the same failure test ($\Delta_W < \tau$, the audited competence margin falling below threshold), yet leave distinguishable signatures. Those differing signatures let us set the response prior (a secondary contribution—detection does not depend on it). Drift is slower and phase-marker-correlated (tracking program-phase transitions) and is *self-consistent* with a new-but- valid regime (the forward model S_{res} stays calibrated). An attack is abrupt, can pulse, is *targeted* at the decision boundary (high S_{res} with small S_{in}), and correlates with a specific co-tenant's schedule whose work yields it no benefit except moving the shared C (the contended shared resource). A small logistic model over the auditor-observable $(\Delta S_{in}, \Delta S_{res}, |\nabla\hat{\Delta}|,$ co-tenant- corr) suffices to separate the two. In the evaluation, probe P3 validates this

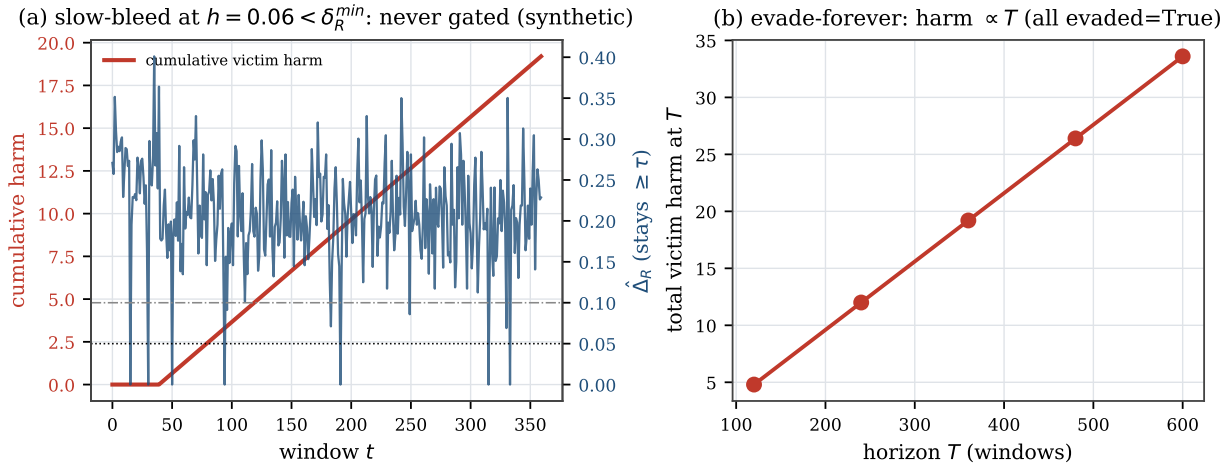


Figure 5: The coverage hole, demonstrated (controlled harness, real Bouncer region dueling). (a) a slow-bleed at $h<\delta_R^{\min}$ accumulates unbounded victim harm while $\hat{\Delta}_R \geq \tau$ and no alarm fires; (b) it evades at every horizon, so total harm $\propto T$. The closed-form floor is $\delta_R^{\min}(B)=k\sigma_R \propto 1/\sqrt{B}$ $(0.19, 0.13, 0.094, 0.066$ for per-pool leader counts $n_{L,R}=2, 4, 8, 16)$: more leader budget shrinks the hole but never closes it.

Table 1: Per-controller instantiation, ordered by set-locality (whether reward and decision share a dueling set). Predictors are set-local by structure; their reseed-identifiability is untested for real predictors; a *synthetic* stateful-predictor model shows a stateful reward *whose contrast scales with the erased state* confounds the reseeded audit, shrinking $\hat{\Delta}$ to most of the gap being lost (only 8% retained) as warmup lengthens, while a state-invariant contrast survives (Figure 14).

Controller	set-local?	r_t proxy	fallback π_0	dueling resource
Hawkeye/Glider (repl.)	**yes**	hit/miss	RRIP/LRU	cache sets
Perceptron branch	yes (PC)	correct	bimodal/gshare	PC slices
Pythia (RL prefetch)	**no**	acc.×timel.	next-line/off	PC buckets
RL mem. scheduler	partial	−lat./+BW	FR-FCFS	banks/chan.

triage model under cross-validation (holding out data to check it generalizes; Section 11); misclassification only mis-selects between equally-safe responses.

9 Per-controller instantiation

Table 1 maps Bouncer onto four controller classes, ordered by *set-locality*. **Cache replacement** leads: its hit/miss reward is attributed to the very set whose eviction it decided, so the dueling $\hat{\Delta}$ is a clean, localized competence signal—the home turf. **Branch/perceptron** predictors are PC-indexed (keyed by program-counter address) and likewise local. **Prefetching** is *not* set-local—a prefetch on set s rewards a different set s', so the faithful reward cannot be dualized (Section 11.13); it is the cautionary boundary. The **memory scheduler** is the hardest: no clean set-sampling on a shared command bus (the single channel to DRAM) forces a *model-based* $\hat{\Delta}$, which weakens Proposition 1.

10 Methodology

Bouncer's guarantees rest on bounded reward, a partitionable resource, and a secret uniform sampler (assumptions A1–A6, Equation (2), Lemma 1 and proposition 1), plus—for the audit to be *faithful*—set-locality and reseed-identifiability (Definition 3 and remark 1); given these, we argue the guarantees are *environment-agnostic*. We therefore validate the *mechanism* in *one* faithful, controllable synthetic harness—instantiated

Table 2: Hypothesis discharge: where each premise is established (E), measured (M), or assumed (A). Reseed-identifiability (Remark 1) is the load-bearing refinement—it holds by construction in the harness and *fails, measured,* on a real cache.

Premise	Discharged
A1 bounded reward $\in [0, r_{\max}]$	E/M: synthetic by construction; real hit/miss $\in \{0, 1\}$.
Reseed-identifiability of the contrast (Remark 1)	A (synthetic: i.i.d. reward, holds by construction); **fails, measured, on a real cache** (Section 11.13: reseeded $\hat{\Delta} \approx 0$ vs fixed-leader 0.23).
A2 *expected* fully-open windows/episode $\leq D$	A (assumed; *approximated* by the Lorden/Siegmund mean delay $H/(K-\Delta)$, not an upper bound); M: P0/P1 (1-window), R-latency sweep, false re-trust bound (Section 11.12).
A3 marginal false-alarm prob. $\leq 1/\mathrm{ARL}_0$	A (assumed; *approximated* by Siegmund's ARL, the mean run length between false alarms); M: P2 ROC (true- vs false-positive curve), sensitivity sweep.
A4 switch transient $\leq c_{sw}$	A (analytical); M: P1 one-window dip $\leq D r_{\max}$.
A5 N_{ep} episodes over T	A (definitional, counts episodes).
A6 secret *uniform* audit subset	E: construction (Section 5); M: PROBING exposure is ϕ_P-uniform, index-independent (Section 11.11: 0.062/0.064 low/high index vs 0.985 for a sorted prefix).
Prop. 1 finite resolution $\delta_R^{\min}(B)$	E: Equation (2); the sub-resolution slow-bleed is the named open hole (M: Figure 5).
Prop. 1 intact secret (leak f=0)	E/M: secrecy ablation (P4), TPR collapses sharply in the f=0.6–0.8 band (the clean-margin crossing), well before f=1.

three ways in Section 11(P3)—that models a learned controller, a safe fallback, benign drift, and three adversaries, and *also* build the same auditor as a real prefetcher module (a component that predicts and pre-loads memory) inside ChampSim (a cycle-accurate CPU simulator), targeting the L1 data cache and run on the SPEC CPU2017 benchmark programs (Section 11.13). *The controlled quantitative claims (latency, floor, mimicry survival, secrecy) come from the harness; the ChampSim integration reports real IPC (instructions per cycle, the systems throughput metric) on a 3-trace SPEC suite as a proof of deployment, not a full sweep.* The harness models the prefetcher setting as the anchor: a resource of n_{sets}=2048 sets; a latent per-decision *stress* $u \in [0, 1]$ (how far the input is pushed toward C's boundary) through which both drift and steering act and which the auditor never observes; controller reward $\mu_C(u) = r_{\max}\,\mathrm{clip}(0.8 - 0.7u)$ collapsing under stress and a flat floor $\mu_0 = 0.5 r_{\max}$; bounded rewards drawn so $\mathrm{Var} \leq r_{\max}^2/4m$ exactly. A linear transfer $\mathrm{IPC} = 0.6 + 1.4\,\bar{r}$ maps mean reward to throughput, which lets us report the systems metrics. *These two maps are not load-bearing for the qualitative guarantees:* Section 11.8 re-runs the floor and both headline P4 results across a family—two collapse shapes (clip, logistic) at two slopes, an alternative IPC map, a shifted fallback floor $q_0 \in [0.4, 0.6]$, and a joint $\mu_C \times$IPC change—and finds the qualitative guarantees invariant; only the exact percentages move. Detection *power* is the one quantity that is not constant, but its behavior is set by the drift signal-to-noise ratio (SNR), not the estimator resolution. For *any* genuine off-policy episode ($\Delta_W < \tau$) the one-sided CUSUM (a sequential change-detector) drift is $K - \Delta_W \geq \gamma/2 = 0.05$, which at the tau boundary is $1.6\,\sigma_\Delta$ on the background/TRUSTED pool (n_{bg}=8, σ_Δ=0.031) and $3.2\,\sigma_\Delta$ only at full duty (n_L=32, σ_Δ=0.016) after Tier-A escalates (per-domain audits see less). So detection is a matter of *latency*, not deadline-free power: every off-policy gap eventually gates, but the fixed-deadline TPR falls as $\Delta_W \to \tau$ (Figure 16: at Δ_W=0.04, TPR= 0.03 by 10 windows but 1.0 by 20). What grows as Δ_W rises toward τ is the detection *latency* $D \approx H/(K - \Delta_W)$, up to $\sim$13 windows. We pin the Section 5 operating point at n_L=n_F=32, m=64 ($|W| \approx 1.3 \times 10^5$ decisions), τ=0.05, K=0.10, H=0.8. Conditions are clean, benign-drift, and three attacks (broad DoS, adaptive mimicry, regional covert); baselines are no-auditor, input-OOD-only (S_{in}), oracle-Δ, and always-fallback. The harness and the ChampSim skeleton are released.

Hypothesis discharge. Table 2 maps every premise of Lemma 1 (A1–A6) and Proposition 1 to where it is established, measured, or assumed—and, crucially, records the *one* premise—reseed-identifiability of the contrast (Remark 1)—that the real-systems study shows *fails* for a real cache, the substance of the set-locality refinement.

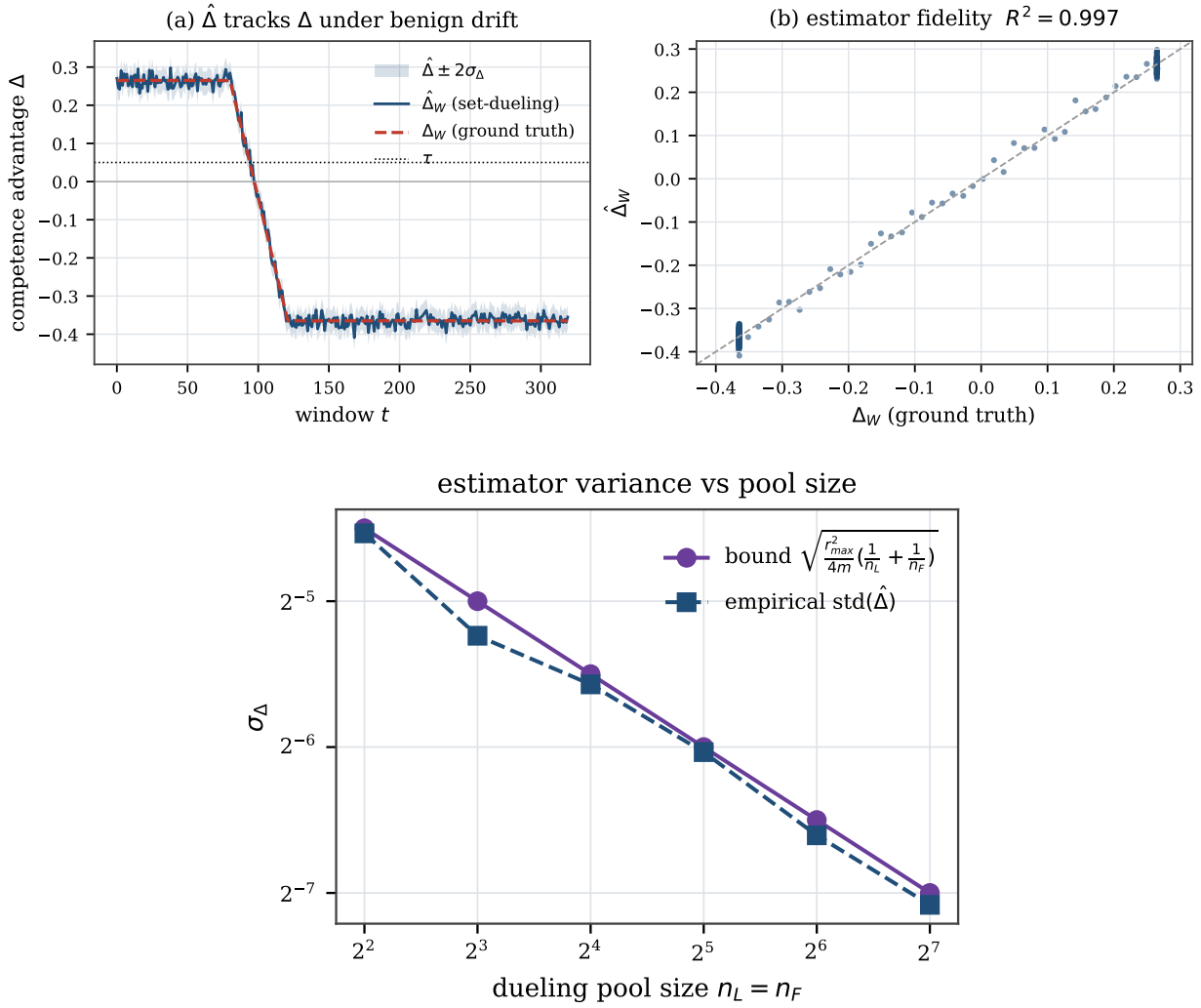


Figure 6: P0. (top) $\hat{\Delta}$ tracks ground-truth Δ (R^2=0.997); (bottom) empirical std($\hat{\Delta}$) matches the bound eq. (2).

11 Evaluation

11.1 P0: the estimator tracks ground truth

The estimator $\hat{\Delta}_W$ tracks the ground-truth competence gap Δ_W closely as a benign drift carries competence from positive through zero to negative, shown in Figure 6(a). The fit is R^2=0.997 with RMSE 0.0141 $\approx$ σ_Δ=0.0156. This is a *controlled-harness* fidelity at the synthetic operating point (m=64); on a real trace with far fewer samples per bucket the estimator is correspondingly noisier (Section 11.13). Figure 6(b) confirms the concentration bound eq. (2): across a pool-size sweep the empirical std of $\hat{\Delta}$ stays just below the bound at 0.85–0.98× it, a tight upper envelope (the bound uses $\frac{1}{4} \geq p(1-p)$).

11.2 P1: the safety floor holds

Under attack, Bouncer holds the controller to the fallback floor and re-trusts once the attack ends; Figure 7 makes the constructive hedge real. Under a poisoning attack (on at window 90, off at 200) the *unguarded* controller crashes far below the fallback floor. Bouncer detects in **one window** here (the worked mean delay

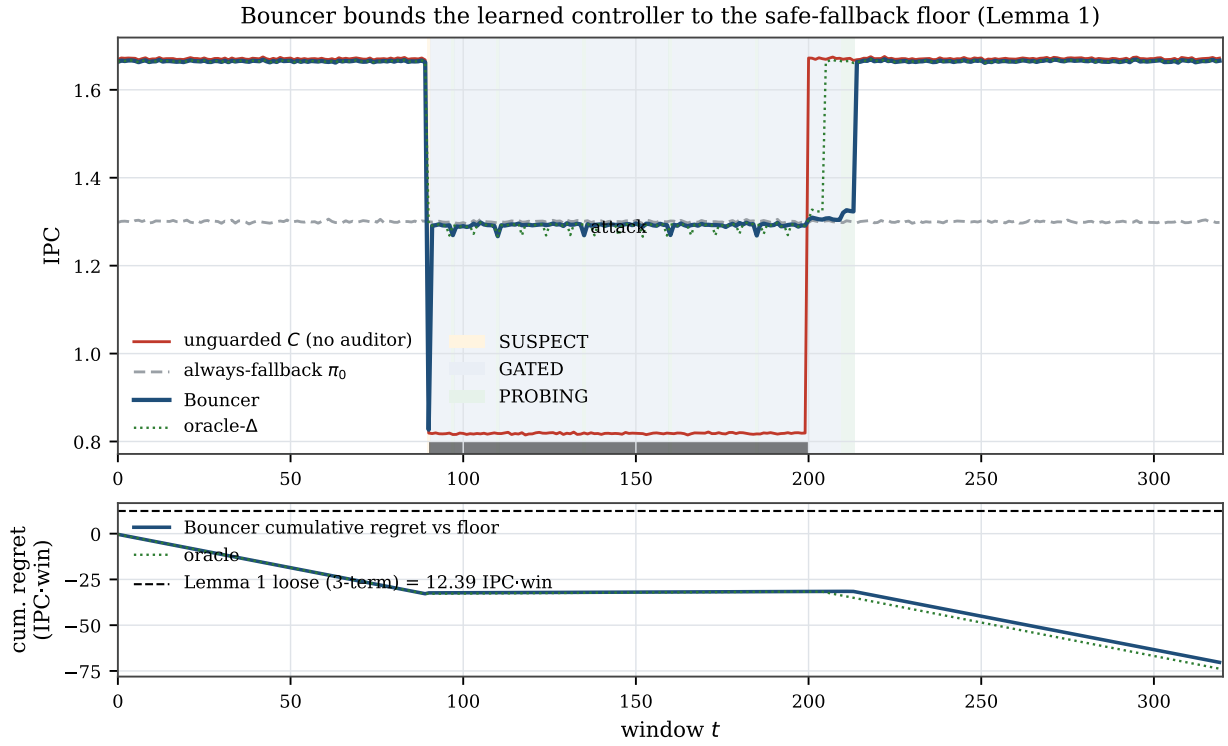


Figure 7: P1. Bouncer bounds the controller to the fallback floor under attack and re-trusts after it ends; the unguarded controller crashes below the floor.

is $D \approx 1.8$ windows; one window is the best case at this high SNR, the signal-to-noise ratio of the duel), floors to π_0, and—crucially—*re-trusts* 14 windows after the attack ends via PROBING, not a fixed timer. Bouncer holds the *steady-state* floor violation to a reported 0.64% (the worst single GATED window; Figure 7), and Lemma 1 *predicts* its mechanism: the audit-exposure term is exactly the Leader-L sets that keep running C under attack, so the *mean* floor violation over the attacked GATED windows is $\phi_G(q_0 - \mu_C^{att})$ slope/IPC$_{\pi_0}$ = 0.579%, matching the measured 0.580% (a design constant forward-predicting the measurement to within 0.002 points); the reported 0.64% is the worst single window, $\sim 2.4\sigma$ of per-window noise above that mean. The only larger excursion is the lone pre-detection window, which dips 36% below the floor: exactly the bounded $\leq D r_{\max}$ term of Lemma 1, and the price of a one-window detection delay. Bouncer recovers performance to 0.55 (returning to the *fallback* floor, not to clean, since the attack genuinely removed C's value), the clean tax stays at 0.9965 (above the 0.99 target; the tax is exactly the n_F Leader-F sets that permanently run π_0), and the bottom panel shows cumulative regret going *negative* (Bouncer beats the floor by capturing C's upside when trusted), the measured worst-case positive excursion staying under the corrected three-term loose bound of Lemma 1 (12.4 IPC·win). Parking the Leader-L sets on π_0 during GATED would zero the exposure term but blind the *measured* re-trust PROBING relies on; we keep the auditable design and pay the reported 0.64%.

11.3 P2: ROC, the §5 chain, and overhead

The competence detector catches every attack while almost never firing on benign behavior, on *both* broad and mimicry attacks: Figure 8, an ROC curve (true-positive rate against false-positive rate), shows it achieving TPR=1.0 (true-positive rate, the fraction of attacks caught) at FPR≤0.05 (false-positive rate, benign runs wrongly flagged), while the input-OOD detector S_{in} works on the broad attack but collapses to TPR=0 under mimicry. Shrinking the dueling pool trades measurement noise for detection latency, tracking the analytic $D \approx H/(K - \Delta)$. Figure 9(a) traces the Section 5 chain: as the pool shrinks from 128 to 2, σ_Δ

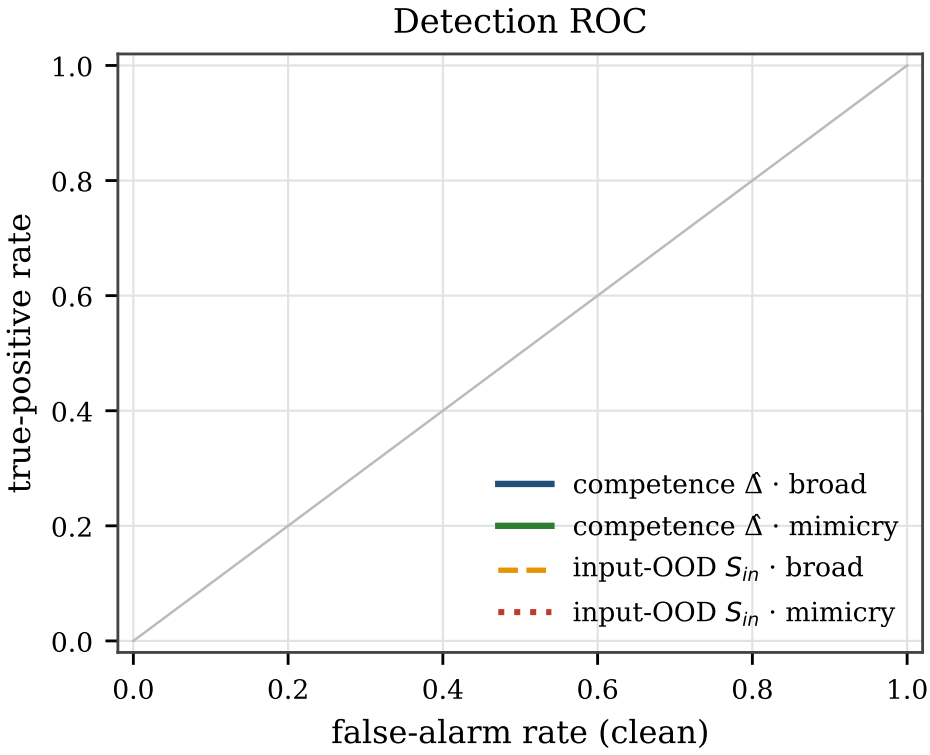


Figure 8: P2 ROC. The competence detector dominates; S_{in} is blind to mimicry.

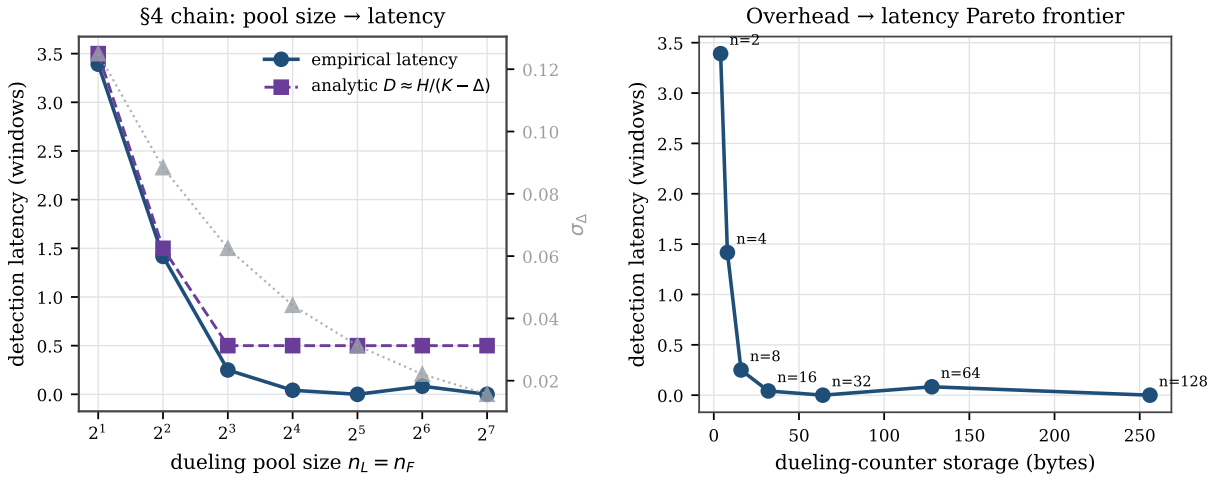
rises $0.016 \to 0.125$, the calibrated H rises $0.05 \to 0.35$, and detection latency rises $0 \to 3.4$ windows. There is a clear knee near $n \approx 8$: above it detection is gap-limited and pool-insensitive, so larger pools only add overhead; Figure 9(b) is the resulting overhead/latency Pareto frontier (best achievable overhead for each latency). The full Section 12 budget is ~ 336 bytes, $\sim 0.13\%$ of a 256 KB SRAM (on-chip static memory) (on-chip static memory), and adds no latency to the cache-access critical path (the timing-limiting sequence of operations) by construction (analytical; RTL timing future work).

11.4 P3: generality and triage

The same auditor wraps three controller classes (Figure 10(a)): a prefetcher and a replacement policy with secret set-dueling, and a memory scheduler with a model-based $\hat{\Delta}$. The steady-state floor holds for all three ($\leq 0.7\%$ violation) under both broad attack and drift; the scheduler pays for its weaker estimator with longer latency and lost mimicry resistance. The auditor also separates attack from drift, so it can label why the floor is threatened (Figure 10(b)). The triage reaches 100% *5-fold cross-validation* accuracy (a standard train/test split into five folds) over 120 episodes on *auditor-observable* features; here the abruptness signal is the slope of the estimated $\hat{\Delta}$, not of ground truth. A drop-one-feature ablation shows the result is robust to removing *any* single feature, including the modeled co-tenant-correlation signal. So the separability comes from the genuine signature gap (gradual input-shift drift vs. abrupt boundary-seeking attack), not from one label-correlated input.

11.5 P4: mimicry survival and the secrecy ablation

This is the result the paper is built around (Figure 11). **(a)** Under an adaptive mimicry adversary that holds S_{in} (the controller's input features) in-distribution while degrading C (its realized competence), the input-OOD-only baseline (a monitor that only watches whether inputs look out-of-distribution) detects *none* of the attacks (0/40 episodes at every strength), while full Bouncer detects *all* (40/40; clean false alarms 0/40



(a) (b)

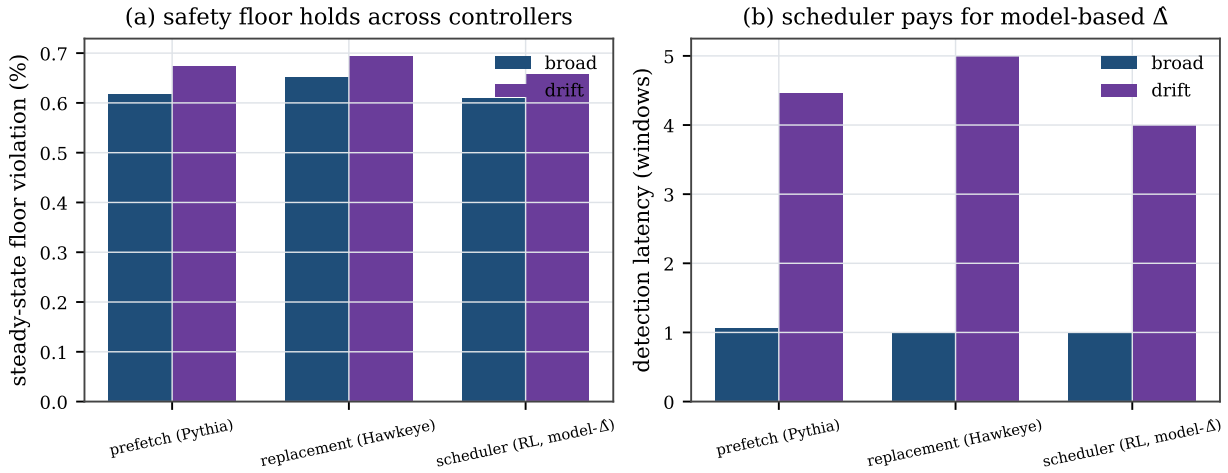
Figure 9: P2. (a) pool size → σ_Δ → H → latency, with a knee near n=8; (b) overhead/latency Pareto frontier.

for both). The input-OOD baseline fails by construction, not by bad luck: its 0/40 is *definitional*, because the mimicry adversary holds S_{in} in-distribution by construction (the harness sets `suppress_features`), so an input-marginal monitor cannot fire; the earned measurements are Bouncer's 40/40 on realized reward and the secrecy ablation below. Across many independent seeds the gap holds with tight confidence intervals (Figure 12(a)): over 50 seeds this is full-Bouncer TPR (true-positive rate, the fraction of attacks caught) 1.0 (Wilson 95% interval, a binomial confidence interval, $[0.93, 1.0]$) vs. input-OOD 0.0 ($[0, 0.07]$), with clean FPR 0.0 ($[0, 0.07]$) for both; detection latency is 1.0 window $[1.0, 1.0]$ (Bouncer fires within one measurement window), the steady floor violation is 0.641% $[0.638, 0.643]$, and the clean tax (the performance cost when nothing is wrong) is 0.99653—all with tight 95% intervals. Measuring *realized competence* rather than input distribution is decisive. **(b)** The secrecy ablation realizes Proposition 1 empirically: against the strongest, redistributive covert adversary, with the dueling pools drawn *within* the victim region (an effective in-region pool of $\sim$4 Leader-L sets, hence a few windows of latency), detection power holds and then falls off a cliff as the secret leaks— TPR=1.0 for leaked fraction $f \leq 0.4$, 0.975 at f=0.6, collapsing to 0.075 at f=0.8 and 0 at f=1.0, with latency rising $1.2 \to \infty$ windows. Panel (a)'s gap over the input monitor is the competence signal itself—an input monitor is blind to a competence drop by construction, so its 0/50 is definitional (Section 11.5); panel (b) is what isolates the role of *secrecy*: this leak-aware regional adversary is caught only while the assignment stays hidden, and detection collapses sharply once the leak crosses the clean-margin threshold (the 0.6–0.8 band; Proposition 1).

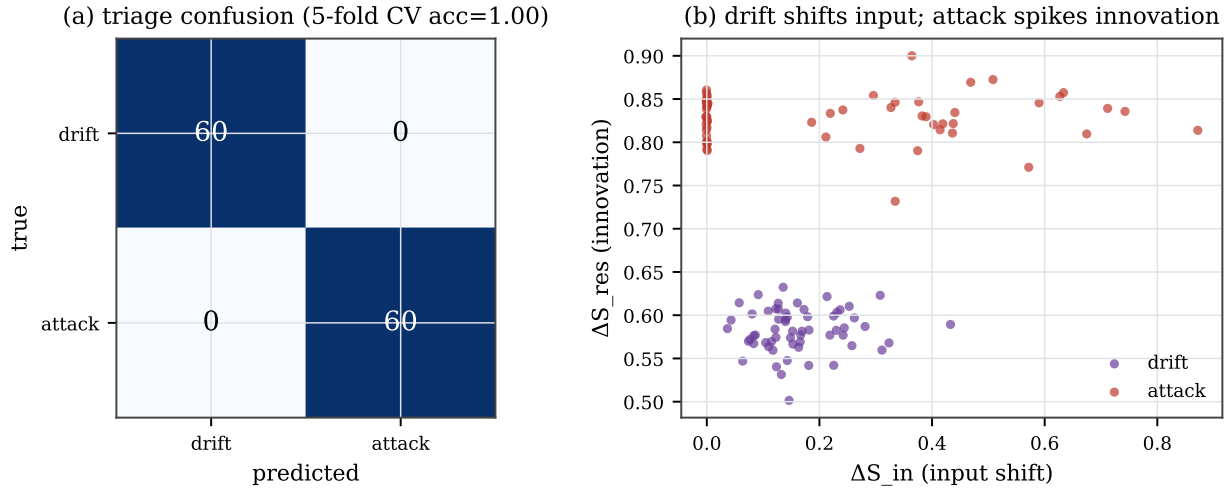
11.6 Robustness: sensitivity and confidence

The headline numbers are not single-seed artifacts (Figure 12(a), N=50, Wilson 95% intervals above), nor a knife-edge operating point: sweeping the trust threshold τ and CUSUM threshold H over a 6×6 grid (Figure 12(b)), most cells achieve $\leq$ 3-window detection with clean FPR $\leq$ 5% and no missed episodes (**28/36**)—a broad robust plateau around the chosen (τ=0.05, H=0.8). Detection degrades gracefully and predictably toward the plateau edges (small H raises false alarms; large τ raises misses), exactly as the Section 5 chain predicts.

Misspecification robustness. Because the harness reward is built from the theorems' assumptions (bounded reward, near-i.i.d. sets)—i.e., a reward specification that may not match reality—we test whether the results survive violating them. We give each set a fixed competence offset $\sim \mathcal{N}(0, \sigma_{het})$ (hot/cold sets, conflict skew), so Leader-C and Leader-F pools are exchangeable only in expectation—the β-bias regime of Proposition 1. Sweeping σ_{het} from 0 to 0.30 (Figure 13), mimicry detection stays at TPR 1.0 and



(a)



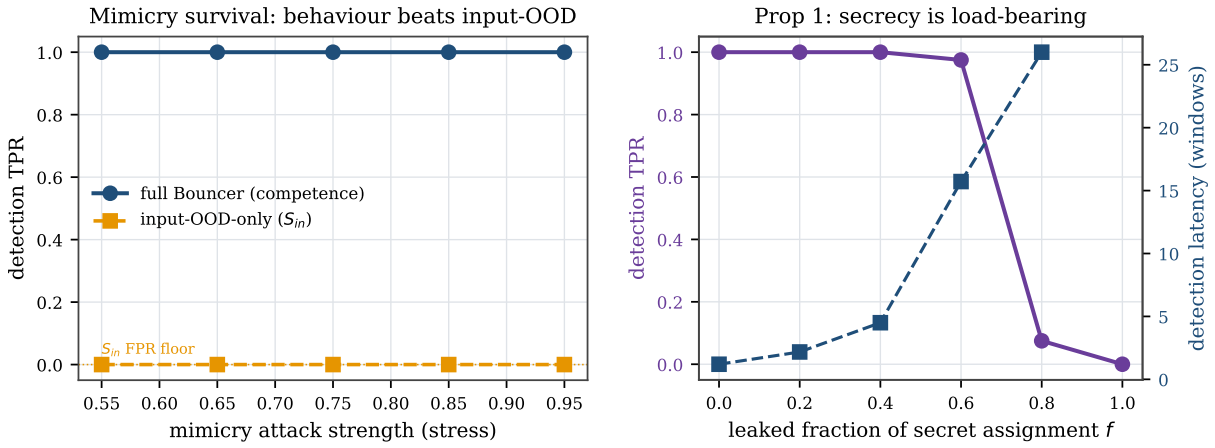
(b)

Figure 10: P3. (a) the floor holds across controller classes; (b) attack-vs-drift triage (acc 1.0).

clean FPR at 0, the steady floor holds (0.60–0.64%), and detection latency degrades only gently (1.0 → 1.7 windows)—because secret *reseeding* re-randomizes which physical sets are leaders each epoch, averaging the heterogeneity bias down. The guarantees are thus not an artifact of the i.i.d. reward model.

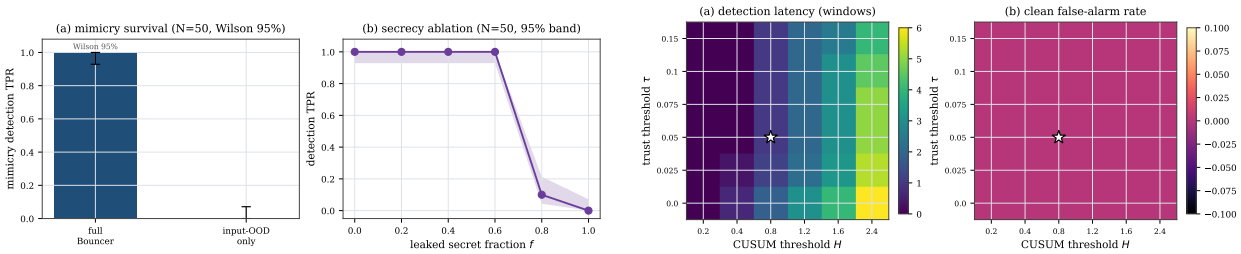
11.7 Reseed-identifiability: a stateful reward that stays auditable

Remark 1 predicts that a secret reseeded audit fails not on *every* stateful reward, but only when the policy *contrast* depends on the state the reseed erases. We isolate the two regimes on a predictor-like controller whose accuracy ramps with consecutive windows on a policy (Figure 14). With *multiplicative* warmup (acc $= \mu(1 - e^{-ck})$, so the contrast scales with the warmup state), the secret per-epoch reseed keeps both pools cold and $\hat{\Delta}$ attenuates from 97% to 8% of the true gap (0.29 → 0.02 of a 0.30 gap over $1/c$=0.3 → 20 windows), while *fixed* leaders recover the full gap (0.30). With *additive* warmup (acc $= \mu - be^{-ck}$, contrast state-invariant at 0.30), the reward is equally stateful yet the reseeded $\hat{\Delta}$ *survives* at 0.30 ($\pm$0.001) across the same warmup range—a stateful reward that stays auditable, the direct counterexample to a statelessness *requirement*. The multiplicative case is the one that matches the ChampSim replacement study on one trace



(a) (b)

Figure 11: P4 (headline). (a) full Bouncer holds TPR=1 under mimicry where input-OOD collapses to 0; (b) detection power vs. the leaked fraction of the secret assignment (Proposition 1).



(a) (b)

Figure 12: Robustness. (a) headline results over 50 seeds with Wilson 95% intervals; (b) sensitivity of detection latency and clean false-alarm rate to (τ, H)—the operating point ($\star$) sits on a broad plateau.

(reseeded $\hat{\Delta}$=0.005 vs fixed 0.23). This is a *synthetic* isolation of the condition, not a real predictor: whether real PC-indexed rewards are reseed-identifiable at window grain remains untested (Table 1).

11.8 Transfer-function sensitivity

A reviewer's natural objection is that the headline percentages are artifacts of the chosen controller-collapse curve $\mu_C(u)$ and IPC (instructions-per-cycle) map. We re-run the floor (P1) and both P4 headlines across a family (Figure 15): the collapse curve as clip and as a logistic at two slopes; the IPC map steeper and concave; a *joint* $\mu_C \times$IPC change; and—the load-bearing axis—the fallback floor q_0 swept over $[0.4, 0.6]$. **The qualitative guarantees are invariant**: across every cell the steady floor violation stays $\leq 0.9\%$ with a genuine drop detected, mimicry TPR is 1.0 for competence vs 0 for the input-OOD monitor, and the secrecy ablation keeps its TPR-vs-f shape; only the exact percentages move (clean tax 0.16–0.51% across maps, on a reduced 20-episode budget per cell). Detection *power* is the one quantity that is not constant. For *any* genuine off-policy episode ($\Delta_W < \tau$) the one-sided CUSUM (a sequential change detector) accumulates a drift $K - \Delta_W \geq \gamma/2 = 0.05$, which at the boundary is $1.6\,\sigma_\Delta$ on the background/TRUSTED pool (n_{bg}=8, σ_Δ=0.031) and $3.2\,\sigma_\Delta$ only at full duty (n_L=32, σ_Δ=0.016) after Tier-A escalation; per-domain audits see less. Every off-policy gap therefore *eventually* gates within the horizon, but the quantity that actually varies is the detection *latency* $D \approx H/(K - \Delta_W)$, up to $H/(K - \tau) \approx 16$ windows at the boundary, so the *fixed-deadline* TPR falls as $\Delta_W \to \tau$. We measure this directly (Figure 16): sweeping the true gap across the off-policy range, the latency rises from 1 to 13 windows tracking $H/(K - \Delta_W)$, and the by-10-window

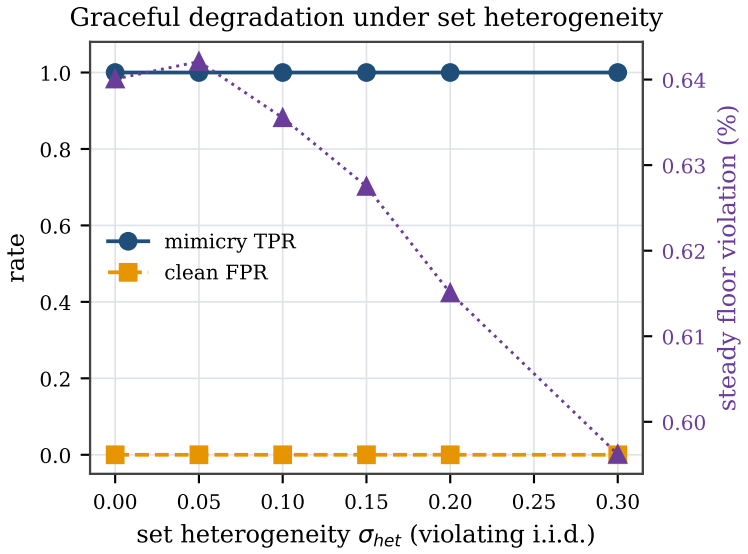


Figure 13: Graceful degradation when the i.i.d.-set assumption is violated: under set heterogeneity σ_{het} up to 0.30, mimicry TPR stays 1.0 and the floor holds; only detection latency drifts up (1.0→1.7 windows).

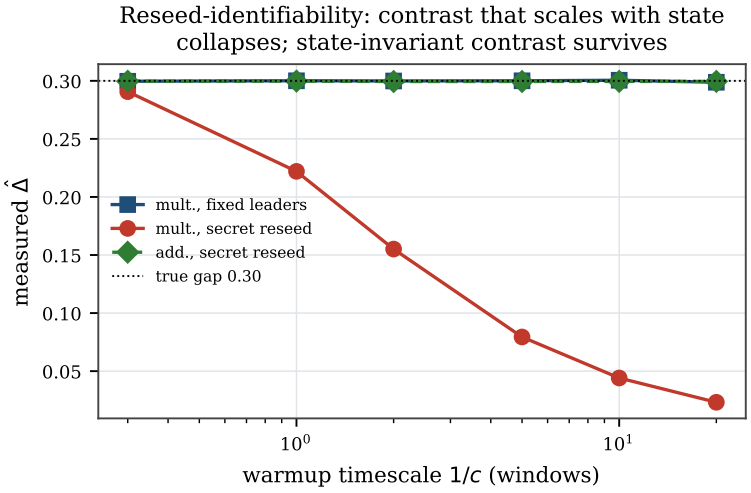


Figure 14: Stateful predictor (accuracy warms up over consecutive windows on a policy). As the warmup timescale $1/c$ lengthens, the secret reseed keeps both pools cold so $\hat{\Delta}$ attenuates toward noise, while fixed leaders recover the true 0.30 gap. This synthetic model *matches* the ChampSim replacement reseed-confound on one trace; it isolates the mechanism but is not a proof of generality to real predictors.

TPR drops from 1.0 at a deep drop to 0.03 at Δ_W=0.04 (recovering to 1.0 by 20 windows). Latency, not deadline-free TPR, is the supported result. *Scope:* this establishes robustness to the *shape* of the collapse curve, the IPC map, and the floor location; the Tier-A feature/confidence maps are not swept (the headline competence-beats-input result is a Tier-B property).

11.9 Adaptive timing adversaries

Two timing-based evasions test threat-model completeness (Figure 17). **Boiling-frog** (Figure 17(a)): a very slow, stealthy degradation tuned to keep the per-window change tiny. It cannot evade—a one-sided

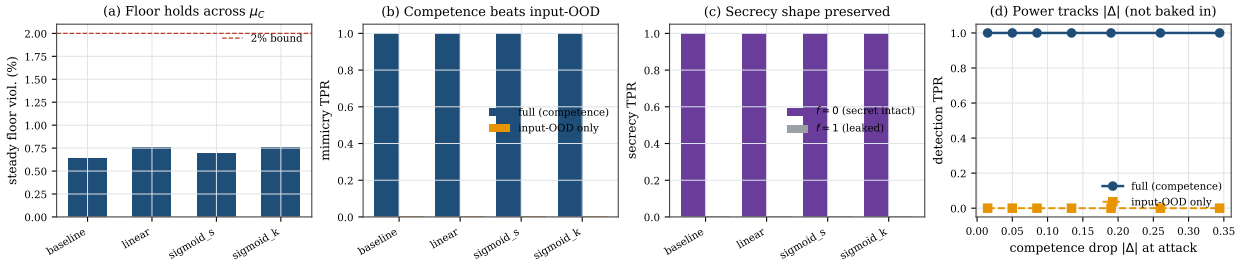


Figure 15: Transfer-function sensitivity. Across collapse-curve shapes (a), the competence-beats-input mimicry result (b) and the secrecy-ablation shape (c) are invariant; (d) the competence detector eventually gates on every off-policy gap (within-episode TPR=1.0 vs. input-OOD 0), because the CUSUM drift $K - \Delta_W \geq 1.6\,\sigma_\Delta$ (background) throughout; what grows toward the τ boundary is the detection *latency* $D \approx H/(K - \Delta_W)$, so the fixed-deadline TPR falls (Figure 16).

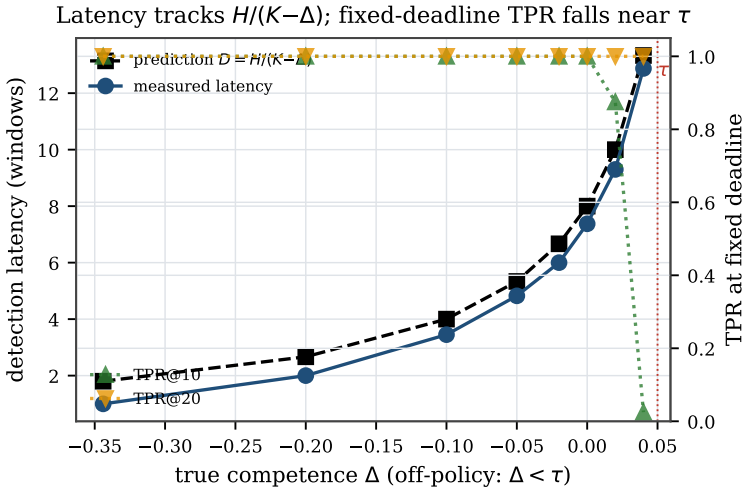
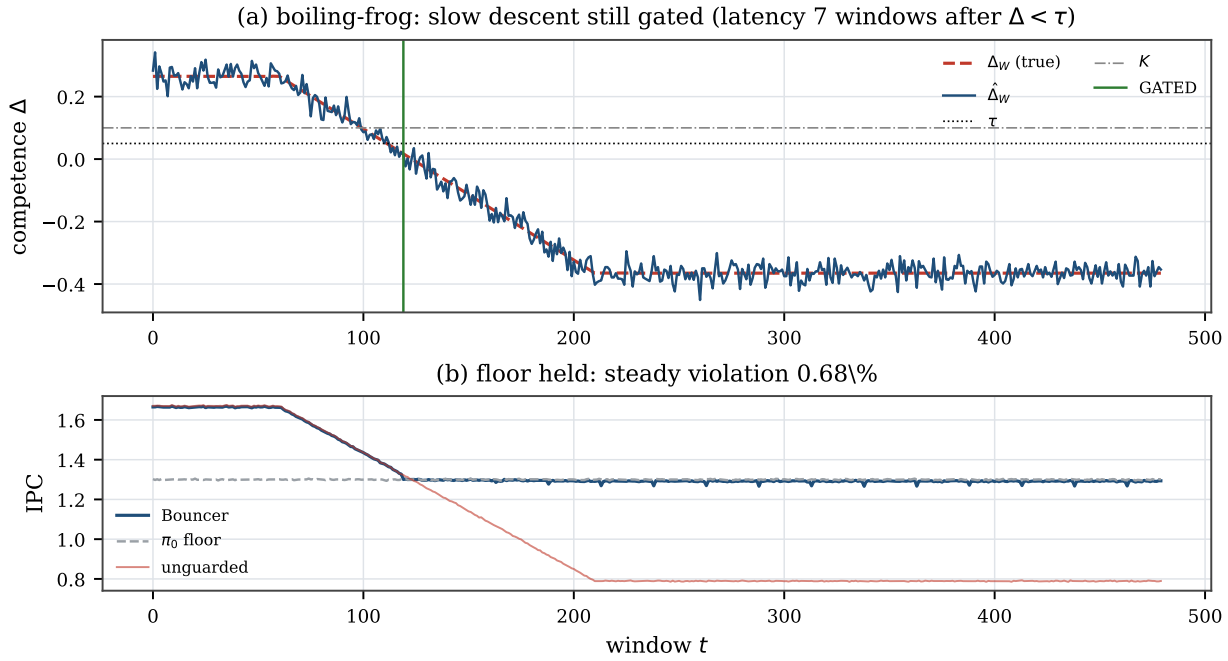


Figure 16: Detection latency vs. competence gap (positive form of Figure 15(d)). Every gap eventually gates (full-duty drift 3.8–$28\,\sigma_\Delta$; 1.9–$14\,\sigma_\Delta$ background), but the detection *latency* rises from 1 to 13 windows toward the τ boundary, tracking the deterministic-drift limit $H/(K - \Delta_W)$, so the fixed-deadline TPR falls (TPR by 10 windows drops to 0.03 at Δ_W=0.04, recovering to 1.0 by 20).

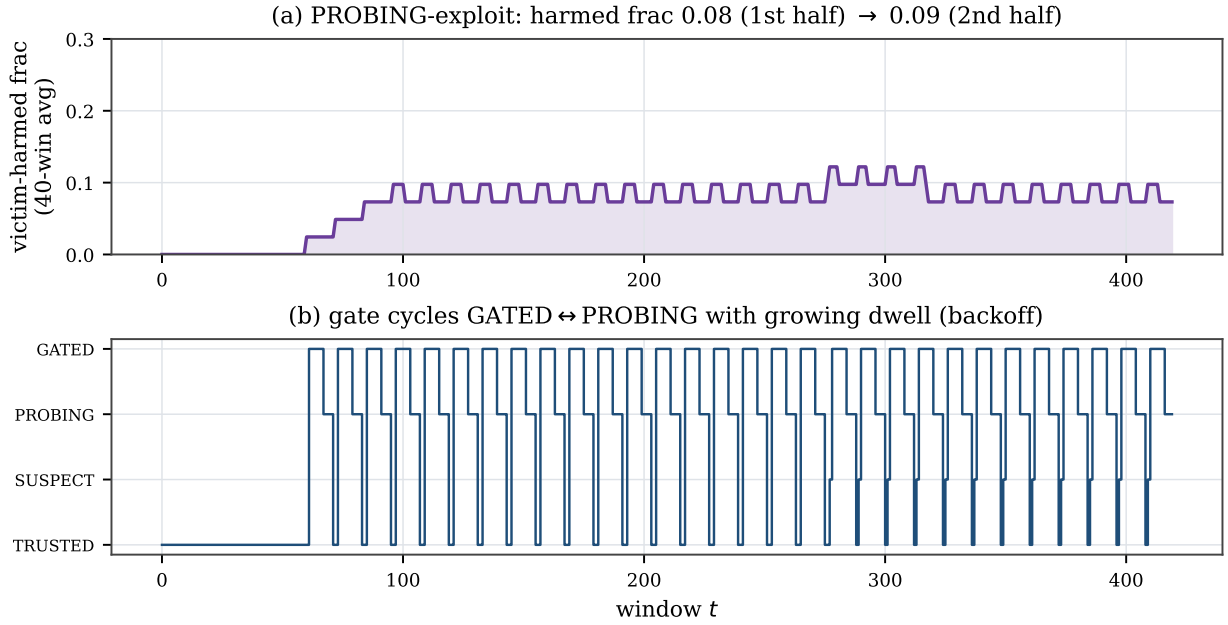
CUSUM (a running-sum change detector) integrates the *level* $(K - \hat{\Delta})$, not the slope, so once $\hat{\Delta} < K$ it accumulates regardless of how slowly it arrived. Bouncer gates 7 windows after the true $\Delta{<}\tau$ crossing (vs. 1 for an abrupt attack); the slow descent only *delays* detection, and the steady floor still holds (0.68%). **PROBING-exploit** (Figure 17(b)): a closed-loop attacker that watches the audit's decision (its gate) and backs off during the GATED/PROBING states to pass the audit, resuming when the TRUSTED state returns—the "wait out the cooldown" strategy. Because re-trust is *measured* (PROBING audits $\hat{\Delta}$) and the dwell backs off exponentially, the victim is harmed in only a small fraction of windows (8.6%) and the attacker spends $\sim$11$\times$ more windows idle than effective; there is no fixed cooldown to wait out.

11.10 Theory validation

Figure 18(a) checks the worked CUSUM constants (CUSUM: a sequential change detector) on an i.i.d. Gaussian signal across a threshold sweep: the empirical ARL_0 (mean time to a false alarm) and detection delay match Siegmund's approximation eq. (6) (geometric-mean ratio 0.998, e.g. ARL_0 empirical 927 vs. theory 938 at $H{=}5\sigma$; a small H-dependent bias remains). This sweep is a *formula sanity check* covering the small-drift, heavy-overshoot regime where the 1.166 correction matters; the deployed detector runs in the



(a)



(b)

Figure 17: Adaptive timing adversaries. (a) boiling-frog is still gated (a CUSUM integrates level, not slope); (b) the PROBING-exploit attacker is bounded to a small harmed-window fraction (8.6%) by measured re-trust + dwell backoff.

opposite, strong-drift regime ($(K-\Delta)/\sigma_\Delta \approx 28$ at n_L=32), where the delay collapses to the deterministic limit $D \approx H/(K-\Delta)$ and ARL_0 is astronomically large (so $\alpha \approx 0$). Figure 18(b) shows that every corrected regret bound envelopes the measured worst-case regret at short attacks, with the tight form hugging the data. It has two panels. Panel (a) sweeps N_{ep} at short attacks (L_{att}=60): the old two-term $N_{ep}Dr_{\max}$, the

corrected three-term loose, and the *tight* form (realized gap $q_0 - \mu_C^{att}$ with ϕ_G/ϕ_P occupancy) all envelope the measured worst-case positive regret ($\alpha T c_{sw} \approx 0$ here); at N_{ep}=6 the two-term reads 15.1, tight 7.8, measured 5.9 IPC·win (IPC: instructions per cycle, a throughput measure), the tight bound hugging the data. Panel (b) sweeps the attack length L_{att}. Because the audit-exposure regret grows linearly in L_{att} while the two-term $N_{ep} D r_{\max}$ is constant, the two-term is *violated* for long attacks (measured 25.8 vs two-term 7.6 at L_{att}=960), whereas the corrected three-term *loose* bound of Lemma 1 continues to envelope. This loose bound is Lemma 1's *expected*-regret guarantee (these single-seed runs sit under it comfortably; it is the mean, not a per-run worst case). On a single persistent drop the two-term is exceeded nearly five-fold (measured 11.9 vs 2.52; `exp_floor_longattack.py`), while the loose three-term (123.6) still dominates. The *tight* form (realized gap and GATED/PROBING occupancy) hugs the measurement at the mean level (12.3 vs 11.9 here); because it uses the *mean* detection delay it is a *mean-level* envelope—it holds in expectation, not with high probability—so weak-drop configurations can exceed it by one to two windows. The loose three-term form is the *expectation* guarantee eq. (4); only its *exposure* term additionally carries a high-probability envelope eq. (5) (the detection and false-alarm terms are bounded in expectation, not with high probability).

11.11 Traffic-weighted exposure: the counterexample, a routing bug, and the repair

The audit-exposure term is traffic-weighted, and A6—a *secret uniform* sampler—is what bounds it in expectation. Three measurements (`exp_floor_traffic.py`, on the real `SetDueling` routing). *The counterexample.* Put all of a window's traffic on one hot set: the ordinary event that the sampler tags it Leader-L (probability ϕ_G) makes the entire window run C, so the realized one-window regret is $r_{\max}$—a 15.6× violation of the naive per-window $\phi_P r_{\max}$ allowance. This is why the bound must be an *expectation*, not a per-window worst case. *A routing bug A6 rules out.* If PROBING audited a *fixed* (sorted-index) prefix of the followers, a low-index hot set would be exposed with probability $1 - n_L/n_{sets} = 0.984$, not ϕ_P—a 15.4× violation of the *expectation* that no secrecy can fix. We found and removed exactly this: the audit subset is now a uniform secret subset (`set_dueling.is_audited`), and a hot set's PROBING exposure is 0.062 at low index and 0.064 at high index, both $\approx \phi_P$=0.064 and index-independent (versus 0.985/0.017 for the sorted prefix). *The repair holds.* With the uniform sampler the expected per-window exposure is $\leq \phi_P r_{\max}$ for *any* traffic (measured per-window mean 0.064 under worst-case concentration), and over independently reseeded windows the exposure total stays under the Hoeffding envelope $\phi_P T_{att} r_{\max} + r_{\max}\sqrt{(T_{att}/2)\ln(1/\delta_s)}$ (empirical coverage 1.00 at δ_s=0.01 for every $T_{att} \leq 480$).

11.12 Fully-open windows per episode: what D actually is

A2's D counts *all* fully-open (C-everywhere) windows in a drop episode—the initial detection delay plus any *false re-trust* excursions, where PROBING re-opens the gate after $T_{reprobe}$ audit estimates above $\tau + \Delta_{hys}$. We measure D *through the real controller* (`exp_retrust.py` drives the actual Tier-B CUSUM and FSM, so re-gating pays the true CUSUM delay, not a one-window surrogate—an error a reviewer caught in an earlier draft). *With the reseeded, near-i.i.d. audit* a sustained true drop yields ≈0 false re-trusts across drop depths (the audit reads $\Delta < \tau$ and rarely strings together $T_{reprobe}$ crossings), so the fully-open fraction is just the initial-detection/SUSPECT transient—0.019 (deep) to 0.067 (near τ), and $D \approx$ the initial delay. This is *not* horizon-independent in general: a temporally-*correlated* (stateful) audit produces runs of crossings, so false re-trust occurs and the fully-open fraction climbs with the correlation ρ—0.008 at ρ=0 to 0.225 at ρ=0.95, again through the real controller. So D is small precisely under the reseed-induced near-independence that reseed-identifiability already requires; a correlated audit inflates it. We therefore state D as a *measured assumption* (A2) for the reseeded regime and name the correlated-audit inflation as a *limitation*, not a bound—and every Lemma-1 numeric floor reported in this evaluation (P1's 12.4; the theory-validation 15.1/7.8/5.9 and long-attack 123.6, 11.9-vs-2.52) uses $D \approx$ the initial delay, valid where re-trust ≈ 0 (as measured here), not for an arbitrary audit.

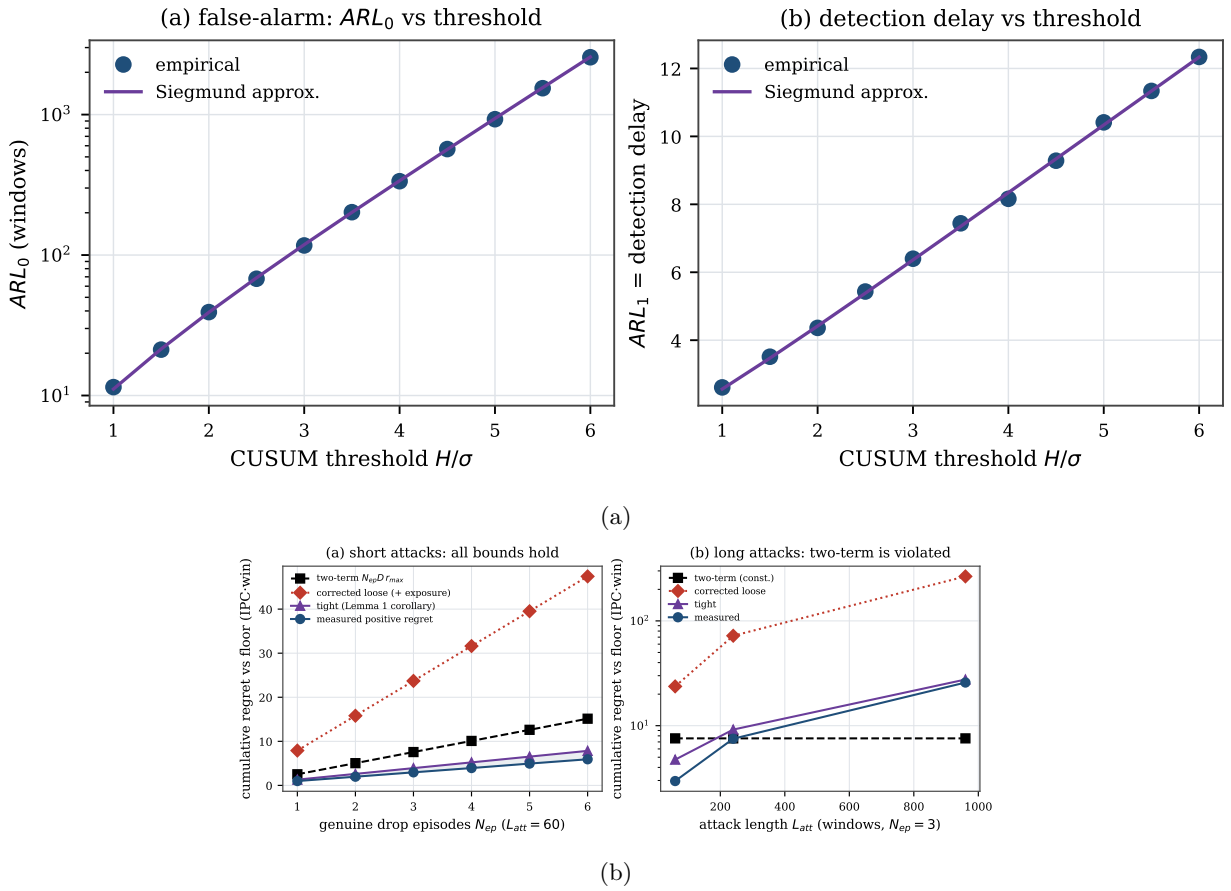


Figure 18: Theory. (a) ARL matches Siegmund; (b) the safety-floor bound envelopes measured regret.

11.13 Real-systems integration: the set-locality boundary

The auditor compiles and runs in real ChampSim as *both* a cache-replacement module (set-local by Definition 3—structurally the clean case, but stateful; see below) and an L1D prefetcher module (*not* set-local). ChampSim is the standard trace-driven CPU cache simulator. The two together test the set-locality *boundary* on real hardware traces: the prefetcher case exhibits the de-localization boundary in detail, and the replacement case shows the clean side is set-local yet stateful—a second boundary (the secret reseed confounds a path-dependent reward), rather than a real-hardware clean-case validation. We present the prefetcher case first because it is where the boundary *bites*—it is the cautionary result, not the headline.

Prefetcher (the cautionary, non-set-local case). We built a real **ChampSim L1D prefetcher module**—the same set-dueling estimator, CUSUM, and gate FSM C++ as Section 12—with the "learned controller" a per-IP stride prefetcher, the fallback prefetch-*off*, the reward the per-set cache-hit rate, and the attack a cache-polluting corruption (the dueling "sets" here are virtual block-address buckets, blk mod 2048, not physical L1D sets). It compiles and runs on SPEC CPU2017 with **no added work on the cache-access critical path by construction** (the confirmer, CUSUM (a running change detector), reseed, and FSM are off the critical path (the timing-limiting logic) and epoch-grained; the per-access cost is only a 2-bit pool-tag lookup and output mux—the same kind of table read DIP/DRRIP already carry; RTL timing is future work, Section 12). The real $\hat{\Delta}$ is far noisier than the synthetic one, so we ran at a looser operating point (only $m \approx 19$ accesses/bucket/window land, so $\hat{\Delta}$ swings in $[-0.8, +0.7]$) (τ=0, H=0.25) than the synthetic (τ=0.05, H=0.8, m=64); we disclose this rather than claim the synthetic R^2=0.997 estimator fidelity transfers (it does not—that is a controlled-harness result).

Table 3: ChampSim L1D suite (9M-insn SPEC). The floor cap bounds the attack on every trace; the cache-hit reward proxy sets the clean tax.

SPEC trace	prefetch benefit	attack loss (unguard)	Bouncer clean tax
600.perlbench (compute)	+1.7%	0.3%	1.7%
619.lbm (streaming)	≈0 (hit-rate)	7.9%	2.3%
654.roms (mem-bound)	+12.7%	5.2%	11.2%

Across a three-trace suite (Table 3, Figure 20) the candid lesson is: the auditor is only as good as its competence reward. The *floor cap is reliable*—on every trace Bouncer ends at the prefetch-off floor, so the corruption attack's worst case is bounded: on streaming 619.lbm a corrupted prefetcher costs the unguarded config a large IPC hit (7.9%, where IPC is instructions per cycle, a throughput measure) while Bouncer (already at the floor) is unharmed; on compute-bound 600.perlbench the attack is nearly harmless (0.3%) and so is the cap. But the per-set cache-hit reward *proxy* mis-estimates prefetch competence, so the clean tax ranges 1.7–11%: on memory-bound 654.roms stride prefetching genuinely helps IPC (1.14 vs the 1.01 off floor), yet the hit-rate signal misses that benefit (it comes from latency/MLP, not raw L1D hit count), so Bouncer over-gates. Two further honesty points: the gate is *competence*-driven, not attack-label-driven—on lbm it engaged before the attack onset, so this is a floor cap, *not* attack-specific detection; and the TRUSTED→GATED transition *dynamics* are shown where competence is controllable (the synthetic Figure 7), since no available naive-prefetcher reward exhibits a clean pre-attack TRUSTED state here.

It is tempting to blame the hit-rate proxy and propose the controller's *own* reward as the fix—but we tested exactly that and it is *insufficient*. Holding $(\tau, H, \text{window}, \text{partition}, \text{FSM})$ fixed and swapping *only* the reward, from per-set cache-hit to the prefetcher's own accuracy×timeliness, moves the roms clean tax only 11.2% → 9.7% and still loses 5% to the attacked-but-ungated prefetcher (single trace, single seed; `hardening/experiments/N1_reward_swap/`). The reason is a **fundamental tension in what set-dueling can measure**: the cache-hit reward is *set-local* (attributable to the dueling set) but a poor utility proxy, whereas the own-accuracy reward is a faithful proxy but *not set-local*—a prefetch triggered on a Leader-L set targets a *different* address that lands on a *different* set, so the dueling mis-credits it and $\hat{\Delta}$ collapses to ≈ 0. The dominant lever is therefore the observation window / signal SNR, not reward fidelity: lengthening the epoch (window 40k→200k) removes the over-gating even with the crude proxy. This is a genuine limitation of set-dueling *for prefetchers*, whose benefit is inherently de-localized; *cache replacement*—where the decision and its hit/miss outcome share a set—is set-local by construction (the paper's "home-turf" condition): the structurally clean case in theory, though Section 11.13 shows a real cache's stateful reward adds a second, reseed-identifiability requirement before that cleanness is measurable. The milestone this real-simulator boundary study sets up is accordingly sharper: a long-epoch, *set-local* competence signal (latency-weighted, replacement-style), not merely "the controller's own reward."

Replacement (set-local but *stateful*—a sharper boundary). The same auditor C++ compiles and runs as a real ChampSim *LLC replacement* module: DRRIP-style dueling on *physical* cache sets (learned C = an aggressive LIP-style insertion, fallback π_0 = SRRIP-HP) scored by the demand-hit reward. Three real findings (Figure 19, 623.xalancbmk), each of which *sharpens* rather than confirms the easy "home-turf" story. *(i) The prior "$\hat{\Delta} \approx 0$" was a software artifact, not sampling.* A ChampSim replacement module's per-access hook fires only if the module also provides a fill hook (`has_cache_fill`); without it the module sees *hits only*, the demand-hit reward collapses to 1, and $\hat{\Delta}$ is *identically* 0. A cache fill is the insertion of a line on a miss, so without that hook the module never sees misses. Adding the fill hook makes $\hat{\Delta}$ a real signal—so the earlier null was a missing callback, not a horizon limit. *(ii) The whole-cache competence gap is real and large:* forcing every set to the learned policy gives a far higher LLC hit rate than the fallback (33.6% vs 4.8%; LLC is the last-level cache). *(iii) Yet the secret per-epoch reseed—the very mechanism that buys mimicry resistance—**confounds** the per-window estimate for this controller.* With reseeding, $\bar{r}_L \approx \bar{r}_F$ and $\hat{\Delta}$ sits at noise (clean mean 0.005) despite that 29-point whole-cache gap; with *fixed* leaders (reseed off, DRRIP's own configuration) the *same* estimator resolves it (clean mean $\hat{\Delta} = 0.23$). The reason is that a replacement policy's reward is *path-dependent*: a set must run one policy *consistently* to accumulate the policy-specific cache state DRRIP's PSEL counter integrates, and reseeding each epoch erases it. **Set-locality is thus**

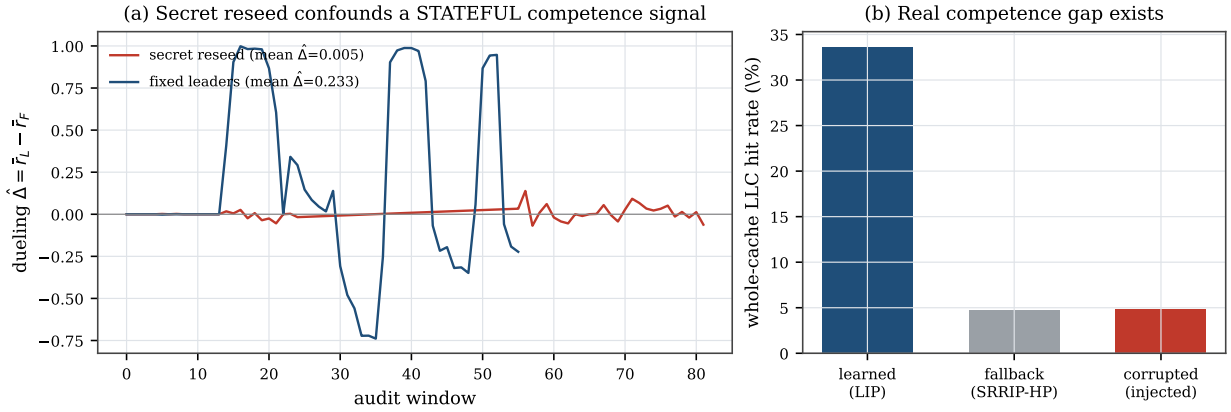


Figure 19: Real ChampSim *replacement* (623.xalancbmk). (a) The secret per-epoch reseed—load-bearing for mimicry resistance—*confounds* the per-window competence estimate for a *stateful* reward: with reseeding $\hat{\Delta}$ sits at noise; with fixed leaders the same estimator resolves the gap. (b) The whole-cache gap is real (learned vs fallback); the `cache_fill` fix makes $\hat{\Delta}$ nonzero (it was identically 0). Set-locality is necessary but not sufficient—a secret, reseeded audit also needs a reseed-identifiable contrast. This is a boundary result; the clean-case dynamics remain in the synthetic harness.

necessary but not sufficient: a secret, reseeded audit additionally requires a *reseed-identifiable* policy contrast (a near-stateless reward is the clean sufficient case)—which the synthetic harness's i.i.d. per-decision reward (A1) satisfies and a real cache, whose contrast is path-dependent, does not at per-window grain. This is a genuine secrecy/measurability tension, not a tuning issue: the clean-case competence and transition dynamics consequently live in the synthetic harness (a faithful *stateless* set-local model, Figure 7), and the deployment milestone is a reseed schedule slow enough to amortize cache-state warmup—a trade-off this result makes precise.

Disclosure (single trace, one seed). The fixed-leader per-window $\hat{\Delta}$ has std 0.57, so the resolution is a *mean-level* effect: the clean mean 0.23 is about 3 standard errors over the $\sim$50 fixed-leader windows of one trace (623.xalancbmk). The gate sat GATED in most clean windows of *both* configurations (89% reseeded, 80% fixed-leader), and the two committed logs follow different protocols (82 windows including the attack for reseeded; 56 clean-only windows for fixed-leader). A multi-trace replication (Section 14) is future work.

12 Overhead budget

The audit adds negligible area and no critical-path latency, as the *analytical* overhead budget in Table 4 shows (an RTL-level measurement is future work). Set-dueling counters reuse the existing ATD (auxiliary tag directory, a shadow tag array)/sampler; the Tier-A sketches and the 16-weight forward model sit in adjacent SRAM and update on a sampled $1/k$ of decisions; the CUSUM detectors are two registers and a comparator each. The budget totals $\sim$336 bytes ($\sim$0.13% of a 256 KB SRAM); we do not model dynamic energy (it scales with the Tier-B duty cycle, but we give no number). Because the confirmer is sampled, off-path, and epoch-grained, it adds **no added latency on the cache-access critical path by construction** (analytically; RTL timing is future work), the non-negotiable constraint. The only per-access work is the 2-bit pool-tag lookup and output mux (a table read comparable to existing DIP/DRRIP set-dueling), not the estimator or gate logic.

13 Related work

Learned microarchitectural controllers. A growing body of work replaces hand-tuned heuristics with online or offline-trained predictors across every major structure Bouncer targets. RL memory schedulers Ipek et al. (2008) and RL prefetchers Bera et al. (2021) learn performance-coupled policies online; learning-

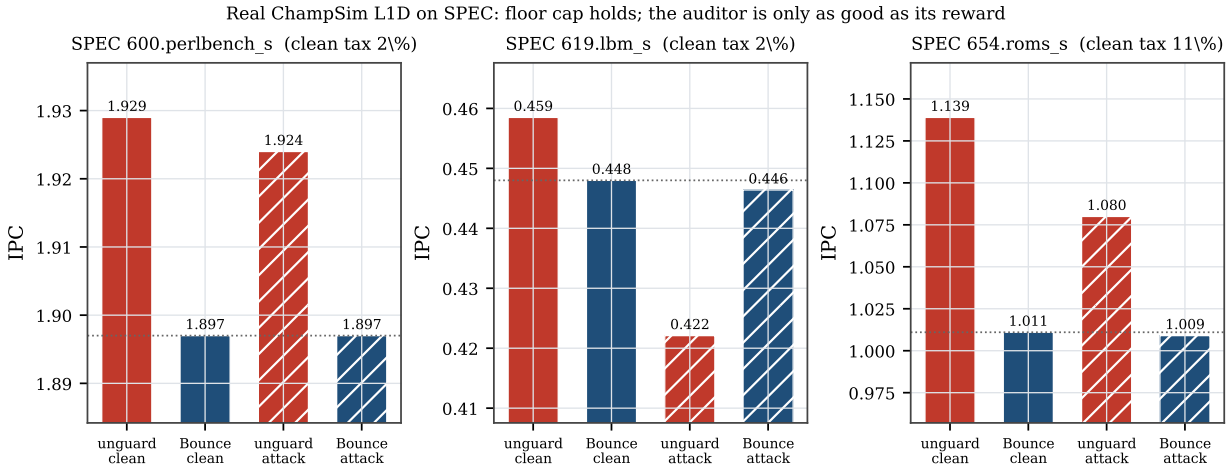


Figure 20: Bouncer as a real ChampSim L1D prefetcher across a SPEC suite. On every trace Bouncer ends at the prefetch-off floor (the cap), bounding the corruption attack's worst case (lbm: 7.9% unguarded loss cannot reach the gated victim). The per-set cache-hit reward proxy sets the clean tax (1.7–11%): roms shows the failure mode—a genuinely-helpful prefetcher under-credited and over-gated. Swapping to the controller's own reward does *not* fix this (only 11.2%→9.7%); a prefetcher's benefit is not set-local, so set-dueling needs a longer epoch or a set-local signal (replacement is the *set-local* case—though a real cache's stateful reward is a further boundary; see Section 11.13).

Table 4: Overhead budget (analytical).

Component	Storage	Per-decision work
Set-dueling counters	64 B (reuse ATD)	reuse existing
S_{in} RP + quantile sketch	224 B	$\leq$ few adds on $1/k$
S_{res} forward model g	32 B	16 MACs on $1/k$
CUSUM detectors ($\times 4$)	16 B	2 add/cmp each
Total	**$\sim$336 B**	off critical path

based cache replacement imitates Belady's optimum with PC-indexed predictors Jain & Lin (2016), distilled neural models Shi et al. (2019), or signature-based hit predictors Wu et al. (2011); and neural and geometric-history branch predictors Jiménez & Lin (2001); Seznec & Michaud (2006) have become the dominant adaptive prediction substrate. These designs share a failure surface: their advantage over a simple baseline (LRU/RRIP, a stride prefetcher, a bimodal predictor) is workload- and phase-dependent and can silently invert under distribution shift. Bouncer is agnostic to which of these controllers it wraps *among the set-local, reseed-identifiable ones it can faithfully audit*; rather than proposing a new policy, it provides, for that class, the missing runtime guarantee that the controller costs, in expectation, no more than a small bounded amount over its own known-safe fallback.

Set dueling and adaptive policy selection. Set dueling Qureshi et al. (2007) dedicates a handful of sampled sets to each of two competing policies and lets a single saturating counter steer the followers, a mechanism extended to re-reference prediction Jaleel et al. (2010), shared-cache thread-aware insertion Jaleel et al. (2008), and evicted-address filtering Seshadri et al. (2012), and rooted in the sampling-monitor methodology of utility-based partitioning Qureshi & Patt (2006) and sampling dead-block prediction Khan et al. (2010); a recent retrospective documents its industrial adoption and limits Qureshi et al. (2023). Bouncer borrows the dedicated-sets-plus-counter substrate but repurposes it from policy *selection* to policy *trust*: where DRRIP duels two heuristics to pick the empirical winner, Bouncer duels the learned controller against its fallback to *certify* a competence advantage and, crucially, keeps the run-C/run-fallback assignment *secret*. Classic set dueling assumes cooperative policies and is content to follow whichever wins; Bouncer

must instead defend against a controller that may be steered or drifting, which is why its dueling sets feed a change detector and a trust gate rather than a preference counter.

Safe RL, runtime assurance, and Simplex. The closest architectural ancestor is Simplex Sha (2001): a simple, formally verified safety controller bounds an unverifiable high-performance one via a decision module. Shielding Alshiekh et al. (2018) synthesizes a correct-by-construction shield from a temporal-logic specification, and constrained- and Lyapunov-based safe RL Altman (1999); Achiam et al. (2017); Chow et al. (2018); Berkenkamp et al. (2017) keep policies inside a certified-safe set, with safe policy improvement Thomas et al. (2015); Laroche et al. (2019) reverting to the data-collecting baseline wherever improvement is not statistically supported (surveyed in García & Fernández (2015)). Bouncer inherits the "use simplicity to control complexity" posture and the bootstrap-to-baseline reflex, but departs in what it must prove. Simplex and shielding require a *verified model of the safety envelope*; constrained RL requires a known cost function. Bouncer needs neither: microarchitectural "safety" is not a state-space invariant but a relative performance floor, so it certifies only that the learned controller's *realized reward* beats the fallback's on randomized dueling sets, with a bounded-regret safety floor (our Lemma) that holds without any reachability proof or verified plant model. The closest relaxation of Simplex's model requirement, Black-Box Simplex Mehmood et al. (2022), still forward-simulates a reachability/admissibility check and defines safety as a state-space invariant; Bouncer instead defines it as a measured relative-competence floor and needs no plant model at all. Concurrent runtime-assurance work certifies recovery *deadlines* for adapting controllers via conformal prediction Shojaei (2026); Bouncer is complementary, certifying a competence floor rather than a recovery time, and adds the secrecy-based mimicry resistance that an observable certificate lacks.

Change detection and OOD monitoring. Bouncer's Tier-B confirmer runs a one-sided CUSUM Page (1954) over its competence signal, inheriting the quickest-detection theory that makes CUSUM minimax-optimal for detection delay under a false-alarm constraint Lorden (1971); Moustakides (1986), with ARL-to-threshold sizing from sequential analysis Siegmund (1985); Tartakovsky et al. (2014). Tier-A tripwires draw on the complementary literature of input-side monitoring: softmax-confidence OOD baselines Hendrycks & Gimpel (2017), conformal validity under exchangeability Vovk et al. (2005); Lei et al. (2018), concept-drift detection Gama et al. (2014), and representation-level two-sample shift tests Rabanser et al. (2019). The essential distinction is what is measured. OOD and drift monitors flag when the *input distribution* moves, which is necessary but not sufficient and, fatally, *mimickable*: an adversary can keep covariate statistics in-distribution while degrading control quality. Bouncer instead measures *realized competence* through secret dueling, so the trigger is the controller's actual reward deficit, not a proxy for it. The closest neighbors to this signal are therefore not input monitors but recent *label-free deterioration* tests: the Suitability Filter Pouget et al. (2025) decides post-deployment fitness from a margin/confidence proxy on the model's own outputs (a $\Delta < \tau$ test); Sequential Harmful Shift Detection Without Labels Amoukou et al. (2024) monitors a proxy-error estimator sequentially with a controlled false-positive rate—structurally our S_{res} plus a one-sided CUSUM; and D3M Nguyen et al. (2025) gives label-free deterioration detection with theoretical false-alarm guarantees, the analogue of our $\sigma_\Delta \to H \to D$ false-alarm sizing. Bouncer differs from all three on axes they share: (i) its per-set dueling estimate is drawn from a *secret, reseeded* sampler, so an adversary who holds the observable proxy in-distribution still cannot game it—the mimicry resistance an observable proxy lacks; (ii) it is a ∼336-byte hardware embodiment with a provable bounded-regret trust gate, not a one-shot software deploy/no-deploy decision; and (iii) it makes explicit the *set-locality* and *reseed-identifiability* conditions that govern *when* a label-free competence proxy is even attributable to the policy that earned it. Input-distribution monitors are the *weakest* of these alternatives—necessary, cheap, and mimickable—which is why Bouncer relegates them to a first-tier gate (S_{in}) and reserves its trust decision for the competence statistic.

Microarchitectural security and co-tenant attacks. The controllers Bouncer audits are also attack surfaces. Contention and timing channels on caches Osvik et al. (2006); Yarom & Falkner (2014); Liu et al. (2015); Gruss et al. (2016b) and occupancy channels Shusterman et al. (2019) leak across cores and VMs in shared clouds Ristenpart et al. (2009), while branch predictors Evtyushkin et al. (2016; 2018) and prefetchers Gruss et al. (2016a); Guo et al. (2022) expose secret adaptive state that an attacker can read out or steer. A learned controller widens this surface: its policy can be poisoned into amplifying leakage or into co-tenant denial of service. Prior defenses harden specific channels; Bouncer is orthogonal and complementary,

bounding the *aggregate* damage any steered controller can do by capping its cost at the vetted fallback. Its mimicry resistance (our Proposition) follows precisely from the secrecy of the set assignment, so an attacker who can observe and game stealthy counter-based detectors Gruss et al. (2016b) still cannot tell which accesses are being scored.

ML-for-systems robustness. Deployed learned components are brittle in exactly the regimes Bouncer guards. Oracle-imitation cache controllers lose accuracy off-distribution Liu et al. (2020), learned indexes degrade on irregular data Marcus et al. (2020), and the Park benchmark documents why RL-for-systems policies behave unlike RL-for-games under noisy, non-stationary workloads Mao et al. (2019); this is the hidden, system-level risk catalogued in Sculley et al. (2015). Production systems respond with bespoke guardrails: SLO-clamped colocation Lo et al. (2015), OOM-bounded autoscaling Rzadca et al. (2020), and fault-triggered fallback in learned software Carbune et al. (2023). Bouncer generalizes these one-off, fault- or threshold-triggered safeguards into a single hardware-cheap mechanism with a *continuous* competence audit and a provable performance floor, applicable to any *set-local, reseed-identifiable* learned microarchitectural controller rather than one tuned guardrail per deployment.

14 Discussion and limitations

Counterfactual cost. Controllers without clean set-sampling (the memory scheduler) force a model-based $\hat{\Delta}$, which weakens Proposition 1; we scope sampling-friendly controllers first. **Redistributive mimicry and the coverage hole.** A global $\hat{\Delta}$ cannot see an attack that preserves the global mean; per-domain dueling raises detection but does *not* fully close it—a slow-bleed adversary below the resolution floor $\delta_R^{\min}(B)$ evades indefinitely (Proposition 1, the coverage hole of Figure 5). We name it rather than hide it. **Ground-truth labeling.** "Off-policy" is operationalized as a sustained competence drop beyond a fixed margin, against which we measure latency. **Overhead vs. sensitivity.** The auditor dies if the Tier-B duty cycle is too high; the minimal-overhead operating point—the knee of Figure 9(a)—is reported as the result, not a footnote. **From simulation to silicon.** The controlled quantitative claims are validated on a faithful harness. The real ChampSim L1D module (Section 11.13) shows the mechanism survives a cycle-accurate simulator and bounds a corrupted prefetcher to the safe floor across a 3-trace SPEC suite; a full SPEC/GAP sweep with a timeliness-aware reward, and the production controllers of Table 1, is the natural next step. **Reward *localizability*, not just fidelity, is decisive.** A reward whose credit lands in the same cache set as the decision is what makes auditing faithful. The per-set cache-hit reward over-gates a genuinely helpful prefetcher on roms (Section 11.13, 11% tax); swapping to the controller's *own* accuracy×timeliness reward does *not* fix it (11.2%→9.7%), because a prefetcher's benefit is intrinsically de-localized (it fetches a different set than it was triggered on) and cannot be dualized to any per-set reward. The structurally clean case is a controller whose decision and outcome share a set—*replacement*—a property of reward *geometry* that sharply scopes where set-dueling competence auditing is faithful.

15 Conclusion

Bouncer turns "is the learned controller still earning its keep?" into one measurable scalar and bounds the cost of the answer. Its organizing result is the *set-locality* condition (Definition 3): a secret per-set competence audit can attribute reward to the policy that earned it only when a controller's reward shares a set with its decision. That makes cache *replacement* the clean set-local case and prefetching the de-localized boundary. The real-systems study adds a second condition: set-locality is necessary but not sufficient, because a real cache is set-local yet *stateful*, so the secret reseed that buys mimicry resistance confounds its per-window measurement—a tension that a *reseed-identifiable* contrast resolves (Remark 1).

For a controller in that class, Bouncer repurposes the set-dueling substrate into a secret competence audit, gates an expensive confirmer behind cheap tripwires, and proves a three-term safety floor—detection, audit-exposure, and false-alarm—tied to the constants of the CUSUM detector. The payoff: for declared domains audited at resolution $\delta_R^{\min}(B)$ with the secret intact, Bouncer bounds the *expected* competence regret, in the controller's own reward, against its vetted fallback, under benign drift and an adaptive mimicry adversary alike, for $\sim$336 bytes and no added datapath latency (analytical). The robustness is neither free nor uncon-

ditional: the competence signal is what beats an input monitor, and its resistance to a *leak-aware* attacker is purchased by the secrecy of the dueling sets—we quantify the price of losing that secrecy, and we name the sub-resolution slow-bleed adversary outside the audited domain as a real, un-closed vulnerability.

Broader Impact

Bouncer is a defensive runtime monitor: it bounds the expected competence cost of a learned microarchitectural controller against a vetted fallback, which serves a co-tenant denial-of-service defense role in shared clouds. Two dual-use considerations. First, the secrecy of the reseeded dueling assignment is what defeats a *leak-aware* attacker (competence measurement is what defeats plain input mimicry); the same primitive could gate a shared controller for censorship-style control, but the mimicry analysis (Proposition 1) also bounds what such gating can hide, since a global $\hat{\Delta}$ cannot conceal a redistributive harm from a per-domain audit. Second, we publish the sub-resolution slow-bleed coverage hole (Figure 5) rather than hide it: naming the resolution floor $\delta_R^{\min}(B) \propto 1/\sqrt{B}$ lets defenders budget leader counts against a target harm rate, which we judge net beneficial because the attack is derivable from the public concentration bound regardless. Online-weight poisoning is out of scope and stated as such (Section 3).

Reproducibility Statement

All synthetic results, figures, and this PDF regenerate deterministically via `./run_all.sh` (CPython 3.11 with the versions pinned in `requirements.txt`; seeded RNG). `scripts/check_paper_numbers.py` (`run_all` stage 9) asserts every headline numeral against the released `results/*.json` to a numerical *tolerance*; we assert tolerance rather than byte-identity because floating-point results shift by 1–2 ulps across numpy builds (the committed `hardening/manifests/` sha256 files are a provenance record of one such build, not a portability claim). The harness invariants (single-assignment-per-window; reseed) are checked by `experiments/test_invariants.py` (stage 0). The real ChampSim studies reproduce via `champsim_plugin/setup_champsim.sh` (prefetcher floor) and `champsim_plugin/run_e1_replacement.sh` (replacement boundary). Lemma 1 has two standalone regressions: `experiments/exp_floor_longattack.py` (the long-attack audit-exposure envelope) and `experiments/exp_floor_traffic.py` (the traffic-weighting counterexample and the expectation/high-probability repair).

Artifact Appendix

The full artifact is released: the auditor and environment (`bouncer/`), one script per evaluation phase (`experiments/`), including the transfer-function sensitivity sweep (`exp_e3_sensitivity.py`, Section 11.8), all result JSON and figures, the real ChampSim L1D prefetcher *and* LLC replacement modules (`champsim_plugin/bouncer_pref/`, `bouncer_repl/`), and the paper source. `./run_all.sh` regenerates every synthetic result, figure, and the PDF in ∼5 minutes on a laptop (CPython 3.11, pinned numpy/scipy/matplotlib/pandas); all RNG is explicitly seeded, so results are deterministic. `champsim_plugin/setup_champsim.sh` clones and builds ChampSim, installs the Bouncer modules, fetches public SPEC CPU2017 traces, and reproduces the prefetcher safety-floor study; `champsim_plugin/run_e1_replacement.sh` reproduces the Section 11.13 replacement boundary study (Figure 19) on `623.xalancbmk` (C++17 toolchain + vcpkg). The standalone auditor additionally builds with `c++ -std=c++17 bouncer.cc -o bouncer_selftest`.

References

Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. In Doina Precup and Yee Whye Teh (eds.), *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pp. 22–31. PMLR, 2017.

Mohammed Alshiekh, Roderick Bloem, Rüdiger Ehlers, Bettina Könighofer, Scott Niekum, and Ufuk Topcu. Safe reinforcement learning via shielding. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI)*, pp. 2669–2678. AAAI Press, 2018.

Eitan Altman. *Constrained Markov Decision Processes*. Chapman and Hall/CRC, 1999.

Salim I. Amoukou, Tom Bewley, Saumitra Mishra, Freddy Lecue, Daniele Magazzeni, and Manuela Veloso. Sequential harmful shift detection without labels. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2412.12910.

Rahul Bera, Konstantinos Kanellopoulos, Anant V. Nori, Taha Shahroodi, Sreenivas Subramoney, and Onur Mutlu. Pythia: A customizable hardware prefetching framework using online reinforcement learning. In *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 1121–1137, 2021. doi: 10.1145/3466752.3480114.

Felix Berkenkamp, Matteo Turchetta, Angela P. Schoellig, and Andreas Krause. Safe model-based reinforcement learning with stability guarantees. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pp. 908–918, 2017.

Victor Carbune, Thierry Coppey, Alexander Daryin, Thomas Deselaers, Nikhil Sarda, and Jay Yagnik. Smartchoices: Augmenting software with learned implementations. *arXiv preprint arXiv:2304.13033*, 2023.

Yinlam Chow, Ofir Nachum, Edgar Duenez-Guzman, and Mohammad Ghavamzadeh. A lyapunov-based approach to safe reinforcement learning. In *Advances in Neural Information Processing Systems 31 (NeurIPS)*, pp. 8092–8101, 2018.

Dmitry Evtyushkin, Dmitry Ponomarev, and Nael Abu-Ghazaleh. Jump over ASLR: Attacking branch predictors to bypass ASLR. In *49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 1–13. IEEE, 2016. doi: 10.1109/MICRO.2016.7783743.

Dmitry Evtyushkin, Ryan Riley, Nael B. Abu-Ghazaleh, and Dmitry Ponomarev. BranchScope: A new side-channel attack on directional branch predictor. In *Proceedings of the 23rd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 693–707. ACM, 2018. doi: 10.1145/3173162.3173204.

João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):44:1–44:37, 2014. doi: 10.1145/2523813.

Javier García and Fernando Fernández. A comprehensive survey on safe reinforcement learning. *Journal of Machine Learning Research*, 16:1437–1480, 2015.

Daniel Gruss, Clémentine Maurice, Anders Fogh, Moritz Lipp, and Stefan Mangard. Prefetch side-channel attacks: Bypassing SMAP and kernel ASLR. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 368–379. ACM, 2016a. doi: 10.1145/2976749.2978356.

Daniel Gruss, Clémentine Maurice, Klaus Wagner, and Stefan Mangard. Flush+flush: A fast and stealthy cache attack. In *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, volume 9721 of *Lecture Notes in Computer Science*, pp. 279–299. Springer, 2016b. doi: 10.1007/978-3-319-40667-1_14.

Yanan Guo, Andrew Zigerelli, Youtao Zhang, and Jun Yang. Adversarial prefetch: New cross-core cache side channel attacks. In *2022 IEEE Symposium on Security and Privacy (SP)*, pp. 1458–1473. IEEE, 2022. doi: 10.1109/SP46214.2022.9833692.

Dan Hendrycks and Kevin Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks. In *International Conference on Learning Representations (ICLR)*, 2017.

Engin Ipek, Onur Mutlu, José F. Martínez, and Rich Caruana. Self-optimizing memory controllers: A reinforcement learning approach. In *Proceedings of the 35th Annual International Symposium on Computer Architecture (ISCA)*, pp. 39–50, 2008. doi: 10.1109/ISCA.2008.21.

Akanksha Jain and Calvin Lin. Back to the future: Leveraging belady's algorithm for improved cache replacement. In *Proceedings of the 43rd International Symposium on Computer Architecture (ISCA)*, pp. 78–89, 2016. doi: 10.1109/ISCA.2016.17.

Aamer Jaleel, William Hasenplaugh, Moinuddin K. Qureshi, Julien Sebot, Simon C. Steely, and Joel Emer. Adaptive insertion policies for managing shared caches. In *Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, pp. 208–219. ACM, 2008. doi: 10.1145/1454115.1454145.

Aamer Jaleel, Kevin B. Theobald, Simon C. Steely, and Joel Emer. High performance cache replacement using re-reference interval prediction (rrip). In *Proceedings of the 37th Annual International Symposium on Computer Architecture (ISCA)*, pp. 60–71. ACM, 2010. doi: 10.1145/1815961.1815971.

Daniel A. Jiménez and Calvin Lin. Dynamic branch prediction with perceptrons. In *Proceedings of the 7th International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 197–206, 2001. doi: 10.1109/HPCA.2001.903263.

Samira Manabi Khan, Yingying Tian, and Daniel A. Jiménez. Sampling dead block prediction for last-level caches. In *Proceedings of the 43rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 175–186. IEEE Computer Society, 2010. doi: 10.1109/MICRO.2010.24.

Romain Laroche, Paul Trichelair, and Rémi Tachet des Combes. Safe policy improvement with baseline bootstrapping. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97 of *Proceedings of Machine Learning Research*, pp. 3652–3661. PMLR, 2019.

Jing Lei, Max G'Sell, Alessandro Rinaldo, Ryan J. Tibshirani, and Larry Wasserman. Distribution-free predictive inference for regression. *Journal of the American Statistical Association*, 113(523):1094–1111, 2018. doi: 10.1080/01621459.2017.1307116.

Evan Zheran Liu, Milad Hashemi, Kevin Swersky, Parthasarathy Ranganathan, and Junwhan Ahn. An imitation learning approach for cache replacement. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pp. 6237–6247, 2020.

Fangfei Liu, Yuval Yarom, Qian Ge, Gernot Heiser, and Ruby B. Lee. Last-level cache side-channel attacks are practical. In *2015 IEEE Symposium on Security and Privacy (SP)*, pp. 605–622. IEEE Computer Society, 2015. doi: 10.1109/SP.2015.43.

David Lo, Liqun Cheng, Rama Govindaraju, Parthasarathy Ranganathan, and Christos Kozyrakis. Heracles: Improving resource efficiency at scale. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA)*, pp. 450–462, 2015. doi: 10.1145/2749469.2749475.

G. Lorden. Procedures for reacting to a change in distribution. *The Annals of Mathematical Statistics*, 42 (6):1897–1908, 1971. doi: 10.1214/aoms/1177693055.

Hongzi Mao, Parimarjan Negi, Akshay Narayan, Hanrui Wang, Jiacheng Yang, Haonan Wang, Ryan Marcus, Ravichandra Addanki, Mehrdad Khani Shirkoohi, Songtao He, Vikram Nathan, Frank Cangialosi, Shaileshh Bojja Venkatakrishnan, Wei-Hung Weng, Song Han, Tim Kraska, and Mohammad Alizadeh. Park: An open platform for learning-augmented computer systems. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pp. 2490–2502, 2019.

Ryan Marcus, Andreas Kipf, Alexander van Renen, Mihail Stoian, Sanchit Misra, Alfons Kemper, Thomas Neumann, and Tim Kraska. Benchmarking learned indexes. *Proceedings of the VLDB Endowment (PVLDB)*, 14(1):1–13, 2020. doi: 10.14778/3421424.3421425.

Usama Mehmood et al. The black-box simplex architecture for runtime assurance of autonomous CPS. In *NASA Formal Methods (NFM)*, 2022. arXiv:2102.12981.

George V. Moustakides. Optimal stopping times for detecting changes in distributions. *The Annals of Statistics*, 14(4):1379–1387, 1986. doi: 10.1214/aos/1176350164.

Viet Nguyen, Changjian Shui, Vijay Giri, Siddharth Arya, Amol Verma, Fahad Razak, and Rahul G. Krishnan. Reliably detecting model failures in deployment without labels. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. D3M; arXiv:2506.05047.

Dag Arne Osvik, Adi Shamir, and Eran Tromer. Cache attacks and countermeasures: The case of AES. In *Topics in Cryptology – CT-RSA 2006, The Cryptographers' Track at the RSA Conference*, volume 3860 of *Lecture Notes in Computer Science*, pp. 1–20. Springer, 2006. doi: 10.1007/11605805_1.

E. S. Page. Continuous inspection schemes. *Biometrika*, 41(1/2):100–115, 1954. doi: 10.1093/biomet/41.1-2.100.

Angéline Pouget, Mohammad Yaghini, Stephan Rabanser, and Nicolas Papernot. Suitability filter: A statistical framework for classifier evaluation in real-world deployment settings. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2505.22356.

Moinuddin K. Qureshi and Yale N. Patt. Utility-based cache partitioning: A low-overhead, high-performance, runtime mechanism to partition shared caches. In *Proceedings of the 39th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 423–432. IEEE Computer Society, 2006. doi: 10.1109/MICRO.2006.49.

Moinuddin K. Qureshi, Aamer Jaleel, Yale N. Patt, Simon C. Steely, and Joel Emer. Adaptive insertion policies for high performance caching. In *Proceedings of the 34th Annual International Symposium on Computer Architecture (ISCA)*, pp. 381–391. ACM, 2007. doi: 10.1145/1250662.1250709.

Moinuddin K. Qureshi, Aamer Jaleel, Yale N. Patt, Simon C. Steely, and Joel Emer. Retrospective: Adaptive insertion policies for high-performance caching. In *ISCA@50 25-Year Retrospective: 1996–2020*, 2023. Retrospective on the ISCA 2007 DIP/set-dueling paper.

Stephan Rabanser, Stephan Günnemann, and Zachary C. Lipton. Failing loudly: An empirical study of methods for detecting dataset shift. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

Thomas Ristenpart, Eran Tromer, Hovav Shacham, and Stefan Savage. Hey, you, get off of my cloud: Exploring information leakage in third-party compute clouds. In *Proceedings of the 16th ACM Conference on Computer and Communications Security (CCS)*, pp. 199–212. ACM, 2009. doi: 10.1145/1653662.1653687.

Krzysztof Rzadca, Pawel Findeisen, Jacek Swiderski, Przemyslaw Zych, Przemyslaw Broniek, Jarek Kusmierek, Pawel Nowak, Beata Strack, Piotr Witusowski, Steven Hand, and John Wilkes. Autopilot: Workload autoscaling at Google. In *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys)*, pp. 16:1–16:16, 2020. doi: 10.1145/3342195.3387524.

D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28 (NeurIPS)*, pp. 2503–2511, 2015.

Vivek Seshadri, Onur Mutlu, Michael A. Kozuch, and Todd C. Mowry. The evicted-address filter: A unified mechanism to address both cache pollution and thrashing. In *Proceedings of the 21st International Conference on Parallel Architectures and Compilation Techniques (PACT)*, pp. 355–366. ACM, 2012. doi: 10.1145/2370816.2370868.

André Seznec and Pierre Michaud. A case for (partially) TAgged GEometric history length branch prediction. *Journal of Instruction-Level Parallelism (JILP)*, 8:1–23, 2006.

Lui Sha. Using simplicity to control complexity. *IEEE Software*, 18(4):20–28, 2001. doi: 10.1109/MS.2001.936213.

Zhan Shi, Xiangru Huang, Akanksha Jain, and Calvin Lin. Applying deep learning to the cache replacement problem. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 413–425, 2019. doi: 10.1145/3352460.3358319.

Alireza Shojaei. Conformal recovery-deadline certificates for runtime assurance of adapting controllers. *arXiv preprint arXiv:2606.25371*, 2026.

Anatoly Shusterman, Lachlan Kang, Yarden Haskal, Yosef Meltser, Prateek Mittal, Yossi Oren, and Yuval Yarom. Robust website fingerprinting through the cache occupancy channel. In *28th USENIX Security Symposium (USENIX Security 19)*, pp. 639–656, Santa Clara, CA, 2019. USENIX Association.

David Siegmund. *Sequential Analysis: Tests and Confidence Intervals*. Springer Series in Statistics. Springer-Verlag, New York, 1985. doi: 10.1007/978-1-4757-1862-1.

Alexander Tartakovsky, Igor Nikiforov, and Michèle Basseville. *Sequential Analysis: Hypothesis Testing and Changepoint Detection*. Monographs on Statistics and Applied Probability. Chapman & Hall/CRC, Boca Raton, FL, 2014. ISBN 978-1-4398-3820-4.

Philip S. Thomas, Georgios Theocharous, and Mohammad Ghavamzadeh. High confidence policy improvement. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, volume 37 of *Proceedings of Machine Learning Research*, pp. 2380–2388. PMLR, 2015.

Vladimir Vovk, Alexander Gammerman, and Glenn Shafer. *Algorithmic Learning in a Random World*. Springer, New York, 2005. doi: 10.1007/b106715.

Carole-Jean Wu, Aamer Jaleel, William Hasenplaugh, Margaret Martonosi, Simon C. Steely, and Joel Emer. SHiP: Signature-based hit predictor for high performance caching. In *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 430–441. ACM, 2011. doi: 10.1145/2155620.2155671.

Yuval Yarom and Katrina Falkner. FLUSH+RELOAD: A high resolution, low noise, L3 cache side-channel attack. In *23rd USENIX Security Symposium (USENIX Security 14)*, pp. 719–732, San Diego, CA, 2014. USENIX Association.